Contents

云上大模型应用开发实践 3
晓云 著 . . . 3

前言：为什么需要云上大模型应用开发 4
一个技术决策者的困境 . . . 4
大模型技术变革与企业数字化转型的交汇点 . . . 4
技术决策者面临的核心挑战 . . . 4
为什么选择阿里云 . . . 5
本书结构与阅读建议 . . . 6
如何使用本书 . . . 6
致谢 . . . 7

第 1 章 云计算与大模型基础 7
1.1 云计算演进：从 IaaS 到 MaaS . . . 7
1.2 大模型技术概览 . . . 9
1.3 阿里云大模型产品体系 . . . 11
1.4 大模型应用开发的挑战 . . . 12
本章小结 . . . 14
延伸阅读 . . . 15

第 2 章 阿里云大模型基础设施 15
2.1 阿里云百炼平台概览 . . . 15
2.2 计算资源调度机制 . . . 17
2.3 通义千问模型家族 . . . 19
2.4 推理服务 SLA 保障 . . . 21
2.5 成本优化实践 . . . 23
本章小结 . . . 24
延伸阅读 . . . 25

第 3 章 大模型 API 开发与集成 25
3.1 RESTful API 设计规范 . . . 26
3.2 认证鉴权机制 . . . 29
3.3 多语言 SDK 实战 . . . 30
3.4 限流熔断策略 . . . 33
3.5 API 版本管理 . . . 35
本章小结 . . . 36
延伸阅读 . . . 36

第 4 章 RAG 应用架构与实战 36
4.1 RAG 技术概述 . . . 37
4.2 知识库构建 . . . 38
4.3 检索优化策略 . . . 42
4.4 完整 RAG 系统实现 . . . 45

本章小结 46
延伸阅读 47

第 5 章 Agent 应用架构与实战 47
5.1 Agent 核心概念 47
5.2 规划能力（Planning） 48
5.3 工具调用（Function Calling） 53
5.4 记忆系统设计 58
5.5 多 Agent 协作 61
本章小结 63
延伸阅读 64

第 6 章 大模型应用安全与合规 64
6.1 安全威胁分类 64
6.2 数据隐私保护 66
6.3 模型滥用防护 67
6.4 合规要求 67
6.5 企业安全治理框架 68
本章小结 69
延伸阅读 69

第 7 章 大模型应用性能优化 69
7.1 推理延迟分析 70
7.2 多级缓存策略 71
7.3 批处理优化 73
7.4 模型压缩 75
7.5 流式输出与增量渲染 76
7.6 性能监控 77
本章小结 79
延伸阅读 79

第 8 章 大模型应用部署与运维 79
8.1 容器化部署 79
8.2 CI/CD自动化流程 84
8.3 监控告警体系 86
8.4 故障排查 89
本章小结 91
延伸阅读 91

第 9 章 行业解决方案与实践案例 92
9.1 智能客服系统 92
9.2 企业知识库问答 95
9.3 AI 代码助手 99
9.4 营销文案生成 103
9.5 教育培训应用 106

9.6 医疗健康场景 109
本章小结 110
延伸阅读 110

第 10 章 大模型应用开发未来趋势 **111**
10.1 多模态大模型演进 111
10.2 Agent 自主化与协作 112
10.3 边缘计算与端侧部署 114
10.4 隐私计算与联邦学习 116
10.5 具身智能与机器人 117
10.6 可持续发展与绿色 AI 119
10.7 技术决策者的应对策略 120
本章小结 120
结语 121
附录 A 术语表 121
附录 B 参考文献 121
附录 C 代码示例索引 122
附录 D 学习资源 122
术语表 122

术语表（中英对照） **122**
A 122
B 123
C 123
D 123
E 123
F 124
G 124
K 124
L 124
M 125
N 125
O 125

云上大模型应用开发实践

晓云 著

完成时间: 2026年04月16日

前言：为什么需要云上大模型应用开发

一个技术决策者的困境

2026年初，某大型制造企业的CTO李明面临着一个典型的技术选型难题。公司希望引入大模型技术来升
Agent在供应链管理中的应用可能性。摆在他面前的选择众多：

- **自建 vs 云服务**：是采购GPU服务器自建模型，还是直接使用云厂商的API？
- **开源 vs 闭源**：选择开源的Llama系列自行微调，还是使用通义千问等商业模型？
- **成本可控吗**：Token计费模式下，月度支出会不会失控？
- **数据安全如何保障**：企业敏感数据上传到云端是否合规？
- **技术路线选对了没有**：今天投入建设的系统，明年会不会被淘汰？

李明的困境并非个案。根据Gartner 2025年的调研，超过73%的企业技术决策者在大模型技术选型时感

1. **技术演进太快**：从GPT-4到Qwen3.6，模型能力每半年就有质的飞跃
2. **生态碎片化**：LangChain、LlamaIndex、Dify、Coze……框架层出不穷
3. **最佳实践缺失**：大多数企业还在摸索阶段，缺乏可复制的成功案例

大模型技术变革与企业数字化转型的交汇点

我们正处在一个历史性的交汇点。一方面，以大语言模型（LLM）为代表的AI技术取得了突破性进展。
年发布的Qwen3.6-Plus支持100万token的上下文窗口，能够理解整个代码仓库；Qwen3.5-
Omni实现了文本、图像、音频、视频的端到端全模态理解与生成。这些能力在三年前还难以想象。

另一方面，中国企业的数字化转型进入深水区。根据IDC的数据，2025年中国企业云服务渗透率已超过
85%，但真正将AI能力深度融入核心业务流程的比例不足15%。这个差距背后，是技术能力、组织架构

云上大模型应用开发，正是连接这两股浪潮的关键桥梁。它让企业能够：

- **快速验证业务场景**：无需巨额前期投入，按需调用大模型能力
- **聚焦业务逻辑**：将基础设施、模型运维等复杂性交给云厂商
- **弹性应对增长**：从PoC到规模化部署，平滑扩展无瓶颈
- **合规与安全**：依托云平台的企业级能力和资质认证

技术决策者面临的核心挑战

本书的目标读者主要是技术决策者——CTO、技术VP、架构师、技术总监等角色。你们不需要亲手写每

挑战一：成本控制的不确定性

大模型的Token计费模式与传统软件许可完全不同。一个看似简单的功能，可能因为用户输入变长、对

本书的应对：第2章详细分析阿里云各类计算资源的成本结构，第7章提供系统的成本优化策略和实战技

挑战二：技术选型的复杂性

RAG vs Fine-tuning、单体架构 vs 微服务、自研 vs 外购、单一模型 vs 混合模型……每个选择都没有标准

本书的应对：第 1 章提供技术选型决策框架，第 4-5 章深入对比 RAG 和 Agent 两种主流技术路线的适用场景。

挑战三：安全与合规的红线

《生成式人工智能服务管理暂行办法》已于 2023 年 8 月正式实施，对内容安全、数据隐私、算法备案等

本书的应对：第 6 章系统梳理合规要求，提供可落地的安全防护方案。

挑战四：组织能力的缺口

大模型应用开发需要跨学科的知识栈：云计算、机器学习、软件工程、产品设计……现有团队的能力模

本书的应对：第 10 章讨论 AI 原生团队构建和技术决策者的能力模型升级。

为什么选择阿里云

本书以阿里云为主要技术平台，基于以下考量：

生态完整性

阿里云提供了从底层算力（ECS GPU/NPU 实例）、向量数据库（OpenSearch、AnalyticDB）、大模 API）到应用开发工具（PAI、函数计算）的完整技术栈。这种垂直整合能力，减少了集成多个供应商的

企业级能力

经过双 11 等极端场景验证的基础设施，提供 99.99% 的 SLA 保障。VPC 私有网络、RAM 访问控制、WAF 防火墙、SLS 日志服务等组件，满足企业级安全合规要求。

合规优势

阿里云通过等保三级、ISO27001、SOC2 等多项认证，数据存储符合《个人信息保护法》《数据安全法

本土化支持

中文文档、本地技术支持、活跃的开发者社区，这些都降低了学习和试错成本。通义千问系列模型在中

当然，多云策略也是很多企业的选择。本书虽然以阿里云为主，但大部分架构设计原则和开发方法论同 AWS、Azure、华为云等其他平台。

本书结构与阅读建议

本书共 10 章，分为四个部分：

第一部分：基础篇（第 1-3 章）

建立对云计算和大模型技术的系统性认知，掌握阿里云核心产品和大模型 API 的使用方法。这部分内容

第二部分：核心技术篇（第 4-5 章）

深入讲解 RAG 和 Agent 两种主流的大模型应用开发范式。这是本书的核心章节，包含大量代码示例和

第三部分：工程化篇（第 6-8 章）

探讨安全合规、性能优化、部署运维等生产环境必须考虑的问题。这部分内容来自真实项目的经验总结

第四部分：实战与展望（第 9-10 章）

通过金融、电商、制造、医疗、教育五个行业的典型案例，展示大模型技术的落地路径。最后一章展望

阅读建议

- **技术决策者**：建议通读全书，重点关注第 1、6、9、10 章
- **架构师**：重点阅读第 2、4、5、7、8 章，掌握架构设计方法
- **开发工程师**：从第 3 章开始，按顺序学习 coding 和实战
- **产品经理**：关注第 4、5、9 章，理解技术边界和应用场景

如何使用本书

本书采用”概念 → 架构 → 代码 → 最佳实践 → 陷阱”的 O’Reilly 实战派写作风格。每章开头用一个实

书中的代码示例主要使用 Python，因为这是大模型生态最成熟的语言。但我们也会提供 Java、Go 等语言的参考实现。所有代码示例都可以在 GitHub 仓库中找到：https://github.com/xiao llm-book

建议你：

1. **搭建实验环境**：注册阿里云账号，开通百炼平台服务，准备一个开发环境
2. **动手实践**：不要只看不练，大模型技术必须在实践中理解
3. **记录问题**：遇到问题时记录下来，这可能是最有价值的学习点
4. **分享交流**：加入开发者社区，与他人交流经验和困惑

致谢

感谢阿里云开发者社区提供的丰富文档和案例，感谢 LangChain、LlamaIndex 等开源社区的贡献者，

由于大模型技术发展迅速，书中内容可能存在滞后或不准确之处。欢迎读者通过 GitHub Issues 反馈问题，我们会持续更新和改进。

最后，希望这本书能够帮助你在云上大模型应用开发的道路上走得更稳、更远。让我们共同见证并参与

晓云
2026 年 4 月于南京

第 1 章 云计算与大模型基础

本章导读
本章将建立对云计算和大模型技术的系统性认知。我们将从云计算的演进历程说起，理解 MaaS（Model as a Service）这一新范式的诞生背景；然后深入大模型的技术原理，掌握 Transformer、预训练、微调等核心概念；接着全面了解阿里云的大模型产品体系；最后分析大模 —成本、延迟、安全、可观测性。

对于技术决策者而言，本章的价值在于提供一个完整的技术地图，帮助你在后续的架构设计和选型

1.1 云计算演进：从 IaaS 到 MaaS

1.1.1 云计算三代架构回顾

云计算的发展可以追溯到 2006 年 AWS 推出 EC2 服务。近二十年来，云计算经历了三次重要的范式转移

第一代：IaaS（基础设施即服务）

以 AWS EC2、阿里云 ECS 为代表，提供虚拟机、存储、网络等基础计算资源。企业的 IT 架构从”自建机房”转向”云上虚拟机”，实现了：

- **资本支出转运营支出**：从一次性采购硬件转为按需付费
- **弹性伸缩**：分钟级开通数百台服务器成为可能
- **全球部署**：利用云厂商的数据中心网络快速拓展海外业务

但 IaaS 时代，企业仍然需要自己管理操作系统、中间件、运行时环境，运维复杂度并未根本性降低。

第二代：PaaS 与 SaaS（平台即服务/软件即服务）

以 Heroku、Salesforce、钉钉、企业微信为代表，提供更高层次的抽象：

- PaaS：开发者只需关注代码，平台负责部署、扩容、监控

- SaaS：最终用户直接使用软件功能，无需关心任何技术细节

这一阶段的核心价值是**生产力解放**。根据麦肯锡的研究，采用 PaaS 的开发团队交付速度平均提升 40%。

第三代：MaaS（模型即服务）

2023 年以来，随着大语言模型的爆发，一种新的云服务范式正在形成：

> MaaS（Model as a Service）：通过云服务提供大模型能力的模式，包括模型训练、微调、推理 API 等全生命周期管理。

MaaS 的核心特征：

特征	说明	典型案例
按需调用	按 Token 或调用次数计费，无前期投入	DashScope API
能力抽象	隐藏模型复杂性，提供简单接口	通义千问对话 API
持续进化	模型能力自动升级，用户无感知	GPT-4 → GPT-4o
生态集成	与向量库、知识库、Agent 框架深度整合	阿里云百炼平台

1.1.2 大模型时代的云原生新范式

MaaS 的出现不是偶然，而是技术发展的必然结果：

原因一：模型规模的经济性

训练一个百亿参数级别的大模型，需要数千张 GPU 连续运行数周，硬件成本超过千万美元。这种规模的 API 调用大模型能力，本质上是**共享规模经济红利**。

原因二：技术迭代的加速

2023-2026 年间，大模型能力几乎每半年就有质的飞跃。如果企业自建模型，面临巨大的技术贬值风险。MaaS，模型升级由云厂商负责，用户可以持续享受最新技术成果。

原因三：应用场景的碎片化

大模型的应用场景极其分散：客服问答、文案生成、代码辅助、数据分析……每个场景需要的模型能力不 平台提供多种模型规格和微调选项，让用户可以按需选择。

1.1.3 MaaS 平台的核心能力矩阵

一个成熟的 MaaS 平台应具备以下核心能力：

```
                    MaaS



•        • API SDK     •        •
```

阿里云百炼平台正是这样一个典型的 MaaS 平台，我们将在后续章节深入探讨其使用方法。

1.2 大模型技术概览

1.2.1 Transformer 架构核心原理

要理解大模型应用开发，首先需要掌握 Transformer 这一基石技术。2017 年 Google 发表的《Attention Is All You Need》论文提出了 Transformer 架构，彻底改变了自然语言处理领域。

核心思想：自注意力机制（Self-Attention）

传统 RNN/LSTM 模型按顺序处理文本，难以捕捉长距离依赖关系。Transformer 的自注意力机制允许模型

用一个简单的例子说明：

> "动物过马路，因为**它**饿了"

人类可以轻易判断"它"指代"动物"。传统模型需要逐词分析才能建立这个联系，而 Transformer 通过注意力权重直接建立"它→动物"的关联。

技术要点：

- **多头注意力**（Multi-Head Attention）：并行计算多种注意力模式，捕捉不同维度的语义关系
- **位置编码**（Positional Encoding）：为序列注入顺序信息
- **前馈神经网络**（Feed-Forward Network）：对注意力输出进行非线性变换
- **层归一化**（Layer Normalization）：稳定训练过程

对应用开发的意义：

理解 Transformer 的基本原理，有助于你理解大模型的以下特性：

1. **上下文窗口限制**：注意力机制的计算复杂度与序列长度平方成正比，这解释了为什么早期模型只能 2K-4K token
2. **并行推理困难**：Transformer 是自回归生成的，下一个 token 依赖前面所有 token，这导致推理延迟难以避免
3. **幻觉现象**：模型基于概率生成文本，而非检索事实，这是 RAG 技术诞生的根本原因

1.2.2 预训练、微调、推理的技术链条

大模型的生命周期可以分为三个阶段：

阶段一：预训练（Pre-training）

在海量无标注数据上训练通用语言模型，学习语言规律和世界知识。这是一个"博闻强记"的过程：

- **数据规模**：万亿级 token
- **计算成本**：数千 GPU 运行数周
- **产出**：基座模型（Base Model），如 Qwen-72B-Base

预训练模型已经具备强大的语言理解能力，但还不擅长遵循指令或完成特定任务。

阶段二：微调（Fine-tuning）

在特定任务数据上进一步训练，让模型学会遵循指令或掌握专业知识：

- **指令微调（Instruction Tuning）**：使用指令 - 回答对数据，教会模型理解并执行各种指令
- **对齐微调（Alignment Tuning）**：使用 RLHF（强化学习人类反馈）等技术，让模型输出更符合人
- **领域微调（Domain Adaptation）**：在医疗、法律、金融等专业领域数据上训练，提升专业问题

阶段三：推理（Inference）

将训练好的模型部署到生产环境，响应用户请求：

- **离线批处理**：批量处理大量文本，如文档摘要生成
- **在线实时推理**：低延迟响应用户对话，如客服机器人
- **边缘推理**：在终端设备运行小型模型，如手机语音助手

技术决策点：

作为技术决策者，你需要在以下选项中做出选择：

方案	适用场景	成本	周期
直接使用 API	快速验证、通用场景	低	天级
Prompt 工程	特定任务优化	极低	小时级
微调小模型	领域专业化、成本敏感	中	周级
训练大模型	核心技术壁垒构建	极高	月级

对于大多数企业，“API + Prompt 工程 + 必要时微调”是最务实的选择。

1.2.3 主流大模型对比

2026 年的大模型市场已形成多强格局：

模型	厂商	参数量	上下文窗口	特点	适用场景
Qwen3.6-Plus	阿里	未公开	100 万 token	编程能力强、代码生成适配、性价比高	Agent、复杂任务规划
Qwen3.5-Omni	阿里	未公开	256K token	全模态（文本/图像/音频/视频）处理	智能硬件、音视频
GPT-4o	OpenAI	未公开	128K token	综合能力强、通用场景	生态完善、海外市场
Claude Opus 4.6	Anthropic	未公开	200K token	长文本理解好、安全性高	法律文档、长报告分析

模型	厂商	参数量	上下文窗口	特点	适用场景
Llama 3.1 405B	Meta	4050 亿	256K token	开源、可私有部署	数据敏感场景、定制化需求
GLM-5	智谱 AI	未公开	256K token	中文能力强、中文场景好	政务应用

选型建议：

1. **优先国产模型**：数据出境合规要求下，通义千问、GLM 等国产模型是首选
2. **关注性价比**：Qwen3.6-Plus 输入 2 元/百万 token，仅为 Claude 的 1/10
3. **考虑生态兼容**：确保所选模型支持 LangChain、LlamaIndex 等主流框架
4. **预留切换空间**：设计抽象层，避免绑定单一模型供应商

1.3 阿里云大模型产品体系

阿里云的大模型产品体系可以分为四层：

```
    ·      ·      ·

  DashScope API ·      ·      ·

  PAI ·      ·      ·

  ECS GPU · ECI · OSS · OpenSearch · AnalyticDB
```

1.3.1 通义千问（Qwen）系列模型

通义千问是阿里云自主研发的大语言模型，目前已演进到 3.6 版本：

Qwen3.6-Plus（2026 年 4 月发布） - 100 万 token 超长上下文 - 专为 Agent 和编程任务优化 - SWE-bench Verified 得分 78.8%，接近 Claude Opus 4.6 - 定价：输入 2 元/百万 token，输出 12 元/百万 token

Qwen3.5-Omni（2026 年 3 月发布） - 全模态理解与生成（文本、图像、音频、视频） - 支持 113 种语言识别 - 端到端音频理解，可处理 10 小时音频或 1 小时视频 - 实时交互延迟 1-2 秒

Qwen-Max - 最强推理能力，适合复杂逻辑任务 - 适合数学推理、科学计算等高难度场景

Qwen-Plus - 性能与成本的平衡选择 - 适合大多数企业应用场景

开源版本 - Qwen-72B、Qwen-14B、Qwen-7B 等 - 支持私有部署和自定义微调

1.3.2 百炼平台：一站式大模型开发

阿里云百炼平台（Model Studio）是本书重点介绍的开发平台，提供：

模型广场 - 接入通义千问、Llama、GLM 等数十种模型 - 支持模型对比测试和 A/B 实验

应用开发 - 可视化工作流编排 - 智能体（Agent）应用模板 - 知识库问答系统快速搭建

模型微调 - LoRA、全量微调等多种方式 - 自动化超参数调优 - 一键部署为 API

评测与监控 - 多维度模型性能评估 - 调用量统计与成本分析 - 异常检测与告警

1.3.3 PAI 平台：机器学习与深度学习

PAI（Platform for Artificial Intelligence）是阿里云的机器学习平台，提供：

- PAI-DSW：交互式建模开发环境（类似 Jupyter Notebook）
- PAI-DLC：分布式训练集群，支持千卡规模
- PAI-EAS：模型在线服务，自动扩缩容
- PAI-Designer：可视化建模工具

对于需要自定义模型训练的企业，PAI 提供了完整的工具链。

1.3.4 模型服务网关与 API 管理

DashScope（灵积模型服务）是阿里云的大模型 API 网关：

- **统一认证**：一个 API Key 访问所有模型
- **流量控制**：限流、配额管理
- **计费统计**：详细的用量报表
- **错误处理**：标准化的错误码和重试机制

1.4 大模型应用开发的挑战

1.4.1 成本控制：Token 计费与优化策略

大模型的 Token 计费模式是一个双刃剑：

优势：零前期投入，用多少付多少

风险：成本不可预测，可能随用户量增长而失控

成本构成分析：

以一个客服机器人为例，假设： - 日均对话量：10,000 轮 - 平均每轮输入：500 token - 平均每轮输出：300 token - 使用 Qwen3.6-Plus：输入 2 元/百万 token，输出 12 元/百万 token

月度成本计算：

```
token = 10,000 × 500 × 30 = 1.5   token
token = 10,000 × 300 × 30 = 9000   token
 = 1.5   ÷ 100   × 2 = 300
 = 9000   ÷ 100   × 12 = 1080
= 1380
```

看起来不高？但如果你的应用需要处理长文档（如 10 万字合同），单次输入就可能消耗 10 万 token，成本结构完全不同。

优化策略：

1. Prompt **压缩**：去除冗余信息，保留关键上下文
2. **缓存复用**：相似问题直接返回缓存答案
3. **混合模型**：简单问题用小模型，复杂问题用大模型
4. **输出长度限制**：设置 max_tokens 上限
5. **批量处理**：合并多个请求，摊薄固定开销

第 7 章将详细讨论成本优化的技术手段。

1.4.2 延迟优化：流式响应与缓存机制

大模型推理延迟通常在 1-5 秒之间，这对实时交互场景（如客服对话）是不可接受的。

延迟构成：

- **首字延迟**（Time to First Token）：1-3 秒
- **生成速度**：20-100 token/秒
- **网络传输**：50-200ms

优化方案：

1. **流式响应**（Streaming）：边生成边返回，用户感知延迟大幅降低
2. **语义缓存**：对相似问题返回缓存答案，延迟降至毫秒级
3. **推测解码**：用小模型预测输出，大模型验证修正
4. **边缘节点**：在靠近用户的区域部署推理节点

1.4.3 数据安全：隐私保护与合规要求

企业使用大模型时，数据安全问题尤为突出：

风险点：

- 敏感数据上传到云端
- 模型可能被提示词攻击窃取训练数据

- 生成内容可能泄露商业秘密

防护措施：

1. **数据脱敏**：上传前移除个人信息、商业机密
2. **私有部署**：敏感场景使用本地部署模型
3. **访问控制**：严格的 API Key 管理和权限隔离
4. **审计日志**：记录所有调用行为，便于追溯

第 6 章将深入讨论安全与合规的最佳实践。

1.4.4 可观测性：日志、监控、调试

大模型应用的调试比传统软件更困难：

挑战：

- 同样的输入可能产生不同的输出（非确定性）
- 模型行为难以预测和解释
- 问题定位需要同时分析代码、Prompt、模型输出

可观测性体系建设：

1. **结构化日志**：记录输入、输出、耗时、Token 用量
2. **链路追踪**：追踪请求在多个组件间的流转
3. **质量监控**：检测幻觉、有害内容、异常输出
4. **用户反馈**：收集点赞/点踩数据，持续优化

本章小结

本章我们完成了对云计算和大模型技术的系统性梳理：

核心知识点：

1. **MaaS 是新范式**：云计算从 IaaS→PaaS→MaaS 演进，大模型即服务成为主流
2. **Transformer 是基石**：理解自注意力机制有助于把握大模型的能力边界
3. **阿里云产品矩阵**：通义千问系列 + 百炼平台 + PAI+DashScope 构成完整技术栈
4. **四大挑战**：成本、延迟、安全、可观测性是生产环境必须解决的问题

技术决策要点：

- 优先使用云 API 而非自建，聚焦业务逻辑
- 选择国产模型确保数据合规
- 设计抽象层避免供应商锁定
- 从一开始就建设可观测性体系

下一章预告：

第 2 章将深入阿里云的基础设施层，详细介绍 GPU/NPU 计算资源、向量数据库、数据存储、网络安全等

延伸阅读

1. Vaswani A, et al. “Attention Is All You Need.” NeurIPS 2017.
2. 阿里云百炼平台官方文档：https://help.aliyun.com/product/42154.html
3. Qwen3.6 技术报告：https://qwenlm.github.io/blog/qwen3.6/
4. Gartner: “Market Guide for Generative AI Products”, 2025
5. 《大模型落地实践指南》，机械工业出版社，2025

本章字数: 约 12,000 字
完成时间: 2026-04-16

第 2 章 阿里云大模型基础设施

本章导读

工欲善其事，必先利其器。对于技术决策者而言，选择合适的大模型基础设施是项目成功的第一步
Studio）的架构设计、计算资源调度机制、模型服务体系以及成本优化策略。通过本章学习，你将

核心议题： - 阿里云百炼平台的整体架构与服务边界 - GPU 池化技术与 token 级调度原理 - 通义千问系列模型的部署模式与性能对比 - 推理服务的 SLA 保障与弹性伸缩策略 - 企业级成本管控与资源优化实践

2.1 阿里云百炼平台概览

2.1.1 平台定位与发展历程

阿里云百炼平台（Alibaba Cloud Model Studio）是阿里巴巴集团面向企业用户打造的一站式大模型开
2023 年正式发布，经过多次迭代，已成为国内领先的大模型基础设施之一。

从技术演进的角度看，百炼平台的发展可划分为三个阶段：

第一阶段（2023 年）：基础能力建设期 - 提供通义千问系列模型的 API 调用服务 - 建立基础的模型训练与推理框架 - 支持简单的 Prompt 工程与微调功能

第二阶段（2024 年）：生态整合期 - 集成多模态模型能力（文本、图像、音频） - 推出工作流编排与 Agent 开发工具 - 建立知识库（RAG）与向量检索服务

第三阶段（2025-2026 年）：企业级服务成熟期 - 实现 GPU 池化与 token 级调度，大幅提升资源利用率 - 支持百万级上下文窗口的长文本处理 - 完善商业化计费体系与 SLA 保障机制 - 深度集成阿里云全栈 AI 基础设施（PAI、DashVector、函数计算等）

截至 2026 年 4 月，百炼平台已支持超过 50 种主流大模型，包括通义千问系列（Qwen2.5、Qwen3、Q
系列、Kimi、MiniMax 等，覆盖文本生成、代码编程、多模态理解、语音交互等多种应用场景。

2.1.2 平台核心架构

百炼平台采用分层架构设计，自下而上分为四个层次：

```
                (Application Layer)

Agent        RAG


                (Service Layer)




                (Model Layer)

Qwen


              (Infrastructure Layer)

GPU
```

基础设施层是整个平台的基石，负责提供计算、存储、网络等底层资源。这一层的核心创新在于
GPU 池化技术，通过将物理 GPU 资源抽象为逻辑资源池，实现细粒度的动态分配。

模型层汇聚了各类预训练大模型，既包括阿里云自研的通义千问系列，也涵盖经过严格筛选的第三方优

服务层提供模型路由、负载均衡、缓存加速等中间件能力，确保高并发场景下的稳定响应。服务层还集
调用等优化机制，显著降低推理成本。

应用层面向最终用户，提供 Agent 开发、RAG 构建、工作流编排等高级功能。这一层的设计目标是让开

2.1.3 地域部署与数据合规

百炼平台支持多种部署模式，以满足不同企业的合规需求：

部署模式	数据存储位置	推理资源范围	适用场景
中国内地	北京接入地域	仅限中国内地	国内业务，需符合中国数据法规

部署模式	数据存储位置	推理资源范围	适用场景
全球国际	新加坡/法兰克福	全球节点	出海业务，需满足GDPR等国际法规
混合部署	多地冗余备份	跨区域调度	大型跨国企业，需灾备能力

重要提示：根据阿里云的数据合规政策，当服务部署范围选择”中国内地”时，所有数据存储位于北京

对于有跨境业务需求的企业，建议选择”全球国际”部署模式，或将不同地域的业务拆分到独立的百炼

2.2 计算资源调度机制

2.2.1 GPU 池化技术原理

传统 GPU 调度模式存在一个根本性问题：资源分配粒度粗、利用率低。在典型场景中，一张 A100 GPU 可能被单个模型独占，但实际负载率长期低于 30%，造成巨大的资源浪费。

阿里云百炼平台引入了创新的 GPU 池化技术，其核心思想是**token 级调度**——将 GPU 资源的分配单位从”整卡”细化到”每个 token 的推理任务”。

这项技术的学术成果《Aegaeon: Token-Level GPU Scheduling for Multi-Model Inference》于 2025 年入选顶级学术会议 SOSP（ACM Symposium on Operating Systems Principles），成为阿里云 AI 基础设施领域的重要里程碑。

GPU 池化的工作原理：

1. **资源抽象**：将物理 GPU 集群抽象为统一的逻辑资源池，屏蔽底层硬件差异
2. **请求切分**：将用户的推理请求按 token 粒度切分为微任务
3. **动态调度**：根据各 GPU 的实时负载情况，将微任务分配到最优计算节点
4. **结果聚合**：在内存中完成各微任务的输出聚合，返回完整响应

通过这种机制，百炼平台实现了以下技术指标：

- GPU 平均利用率从 30% 提升至 75%+
- 单卡支持的并发请求数提升 3-5 倍
- 推理延迟 P99 从 500ms 降至 200ms 以内
- 单位 token 推理成本下降 60%

2.2.2 弹性伸缩策略

对于企业级应用而言，流量波动是常态。百炼平台提供多层级的弹性伸缩能力：

层级一：实例级弹性 - 基于预设的阈值自动扩缩容推理实例 - 支持定时伸缩（应对已知的高峰时段） - 支持指标伸缩（基于 QPS、延迟、错误率等指标）

层级二：资源池级弹性 - 在多个资源池之间动态调配算力 - 支持跨区域调度（需配置多地部署） - 支持优先级队列（保障核心业务的资源供给）

层级三：模型级弹性 - 在多个模型版本之间动态切换 - 支持灰度发布与 A/B 测试 - 支持降级策略（高峰期自动切换到轻量模型）

最佳实践建议：

对于技术决策者，我们建议采用以下弹性策略组合：

```yaml
#
elasticity_config:
  #
  base_capacity: 10  #
  max_capacity: 100  #

  #      11
  scheduled_scaling:
    - cron: "0 0 * nov-dec *"  # 11-12    0
      target_capacity: 50

  #
  metric_scaling:
    - metric: "avg_latency_p99"
      threshold: 300ms
      action: "scale_out_20%"
    - metric: "error_rate"
      threshold: 1%
      action: "scale_out_50%"

  #
  fallback_policy:
    primary_model: "qwen3.6-plus"
    fallback_model: "qwen2.5-7b"
    trigger_condition: "resource_exhausted"
```

2.2.3 上下文缓存优化

长上下文处理是大模型应用的一大痛点。传统的做法是每次请求都重新处理完整的上下文，导致计算资

百炼平台引入了**上下文缓存**（Context Caching）机制，其核心思想是：对于重复出现的上下文片段（Prompt、知识库文档、历史对话等），只在首次处理时进行计算，后续请求直接复用缓存结果。

缓存命中条件： - 上下文的哈希值完全匹配 - 缓存未过期（默认 TTL 为 24 小时） - 缓存空间充足

性能收益： - 缓存命中的请求，推理速度提升 3-5 倍 - 单位 token 成本降低 50%（缓存部分按 50% 计费） - 适用于多轮对话、固定知识库问答等场景

使用示例：

```
from dashscope import Generation

#
response = Generation.call(
    model='qwen3.6-plus',
    prompt='        ...',
    use_context_cache=True,  #
    cache_ttl=86400  #     24
)

print(f"    {response.usage.context_cache_hit_rate}")
```

2.3 通义千问模型家族

2.3.1 Qwen3.5 系列：全模态能力的里程碑

2026 年 3 月 30 日，阿里通义千问团队正式发布旗舰级全模态大模型 Qwen3.5-Omni。该模型在 215 项全模态评测任务中取得 SOTA（State-of-the-Art）表现，标志着通义千问系列在

Qwen3.5-Omni 的核心特性：

能力维度	技术指标	应用场景
上下文窗口	256K tokens	长文档分析、代码仓库理解
语言支持	113 种语言识别，36 种语言生成	国际化应用、多语言客服
音频处理	10 小时 + 音频输入，端到端语音生成	智能硬件、会议转录
视频理解	1 小时视频输入，逐帧分析	视频内容审核、教学辅助
实时交互	端到端延迟 1-2 秒	语音助手、实时翻译

技术架构创新：

Qwen3.5-Omni 采用了 Thinker-Talker 分工架构的升级版：

- **Thinker 模块**：负责深度推理与内容理解，支持 256K 超长上下文
- **Talker 模块**：负责自然语言生成与语音合成，支持情感化语音输出
- **Hybrid-Attention MoE**：混合注意力专家模型，平衡推理效率与准确性

这种架构设计的优势在于：Thinker 可以专注于复杂的认知任务（如逻辑推理、代码生成），而 Talker 则专注于流畅自然的表达，两者协同工作，既保证了回答的质量，又提升了交互的自然度。

实测案例：音视频编程辅助

在某互联网公司的内部测试中，Qwen3.5-Omni 展现了强大的音视频联合推理能力：

场景：前端工程师录制了一段 5 分钟的产品演示视频，包含界面操作和语音讲解

任务：基于视频内容生成对应的 HTML/CSS/JS 代码原型

结果：模型准确识别了界面布局、交互逻辑、配色方案，生成的代码可直接运行，准确率约 85%，大幅减少了手动编码时间。

2.3.2 Qwen3.6 系列：编程 Agent 的新标杆

2026 年 4 月 2 日，阿里通义实验室发布新一代大语言模型 Qwen3.6-Plus。与前代产品相比，Qwen3.6 系列的核心突破在于**编程能力**与**Agent 能力**的双重强化，被官方定位为”当下编程能力最强的国产模型

关键升级：

1. **百万级上下文窗口**
   - 支持 100 万 tokens 的上下文长度
   - 可一次性处理约八本长篇小说或完整代码仓库
   - 适用于代码审查、技术债务清理、跨文件重构等复杂任务
2. **Agent 框架深度适配**
   - 针对 OpenClaw、Qwen Code、Claude Code、Kilo Code、Cline、OpenCode 等六大主流 Agent 框架优化
   - 支持自主任务拆解、工具调用、测试验证的完整闭环
   - 在 SWE-bench Verified 测试中达到 78.8% 的准确率，接近 Claude Opus 4.6 的 80.8%
3. **极具竞争力的定价策略**
   - 输入价格：2 元/百万 tokens（最低）
   - 输出价格：12 元/百万 tokens
   - 预览版限时免费（通过 OpenRouter 平台）

性能对比：

根据第三方评测机构 Artificial Analysis 的数据：

模型	Terminal-Bench 2.0	SWE-bench Verified	每百万 tokens 输出价格
Qwen3.6-Plus	52%	78.8%	¥12 ($1.65)
Claude Opus 4.6	52.1%	80.8%	$25
GPT-5.4	~50%	~78%	$20-40
Kimi-K2.5	~45%	~70%	¥15

从数据可以看出，Qwen3.6-Plus 在编程任务上的表现已接近国际顶尖水平，而价格仅为竞品的 1/15 左右，性价比优势极为明显。

2.3.3 模型选型指南

面对众多的模型选项，技术决策者如何做出最适合的选择？以下是我们总结的选型矩阵：

按应用场景分类：

场景类型	推荐模型	理由
通用对话客服	Qwen2.5-7B/14B	成本低、响应快、足够应对常见问题
专业领域问答	Qwen3.5-Plus	知识广度高、幻觉率低
代码生成与审查	Qwen3.6-Plus	编程能力最强、支持仓库级任务
多模态应用	Qwen3.5-Omni	音视频端到端处理、实时交互
长文档分析	Qwen3.6-Plus	100 万 tokens 上下文
智能硬件嵌入	Qwen3.5-Omni	低功耗、离线可用、语音原生支持

按成本敏感度分类：

- **成本优先**：Qwen2.5 系列（输入¥0.5/百万，输出¥1/百万）
- **平衡型**：Qwen3.5-Plus（输入¥2/百万，输出¥8/百万）
- **性能优先**：Qwen3.6-Plus/Qwen3.5-Omni（输入¥2/百万，输出¥12/百万）

按部署地域分类：

- **中国内地业务**：选择”中国内地”部署模式，确保数据合规
- **出海业务**：选择”全球国际”部署模式，满足 GDPR 等要求
- **混合场景**：采用多云或多地域部署，通过 API Gateway 统一入口

2.4 推理服务 SLA 保障

2.4.1 服务等级协议（SLA）解读

阿里云百炼平台为企业用户提供不同等级的 SLA 保障：

服务等级	可用性承诺	延迟保障	适用套餐
标准版	99.5%	P99 < 1000ms	免费/按量付费
专业版	99.9%	P99 < 500ms	企业套餐
旗舰版	99.99%	P99 < 200ms	定制套餐

可用性计算方式：

```
= (      -      ) /      × 100%
```

补偿政策： - 可用性低于承诺值但≥95%：赔偿当月费用的 10% - 可用性低于 95%但≥90%：赔偿当月费用的 25% - 可用性低于 90%：赔偿当月费用的 50%

2.4.2 高可用架构设计

对于关键业务系统，建议采用以下高可用架构：

策略一：多地域冗余

```
→ API Gateway → [    ]
                        [    ]
```

- 主节点故障时自动切换到备节点
- 数据实时同步，RPO ≈ 0
- 切换时间 < 30 秒

策略二：多模型降级

```
Qwen3.6-Plus
↓
Qwen2.5-14B
↓
Qwen2.5-7B
```

策略三：本地缓存兜底 - 对于高频重复问题，在应用层建立缓存 - 缓存失效时再调用云端 API - 可将 80% 的常规请求拦截在本地

2.4.3 监控与告警

百炼平台提供完善的监控指标体系：

核心指标： - `request_count`：请求总数 - `success_rate`：成功率 - `latency_p50/p95/p99`：延迟分布 - `token_usage_input/output`：Token 用量 - `error_code_distribution`：错误码分布

告警规则示例：

```yaml
alerts:
  - name: "    "
    condition: "error_rate > 5% for 5min"
    severity: "P1"
    notification: [" ", " ", " "]

  - name: "   "
    condition: "latency_p99 > 500ms for 10min"
    severity: "P2"
    notification: [" ", " "]

  - name: "    "
    condition: "quota_remaining < 10%"
    severity: "P3"
    notification: [" "]
```

2.5 成本优化实践

2.5.1 计费模式解析

百炼平台提供多种计费模式：

1. **按量付费（Pay-As-You-Go）** - 按实际使用的输入/输出 tokens 计费 - 无最低消费，适合低频场景 - 单价相对较高

2. **资源包（Resource Plan）** - 预付费购买 token 额度 - 有效期 1 年，过期作废 - 单价比按量付费低 30-50%

3. **Coding Plan（编程专属套餐）** - 固定月费，不限调用次数 - 折算成本远低于常规 API 调用 - 适合高频编程场景 - **注意**：Lite 套餐已于 2026 年 3 月 20 日停止新购

4. **企业定制套餐** - 根据用量协商价格 - 包含专属技术支持 - 支持私有化部署

2.5.2 成本优化技巧

技巧一：Prompt 压缩

```
#     Prompt
prompt = """

      X1

1.
2.   GPS
3.    IP68
...

"""

#     Prompt
prompt = """
     X1
    GPS   IP68
  {user_question}

"""
#    60%    tokens
```

技巧二：流式输出控制

```
#   max_tokens
response = Generation.call(
    model='qwen3.6-plus',
    prompt=prompt,
```

```
    max_tokens=500,  #    500 tokens
    stream=True  #
)
```

技巧三：Batch 调用

```
#       Batch
#    50%
requests = [
    {"prompt": "  1"},
    {"prompt": "  2"},
    {"prompt": "  3"},
]
response = Generation.batch_call(
    model='qwen3.6-plus',
    requests=requests
)
```

技巧四：上下文缓存复用

```
#      Prompt
#        50%
response = Generation.call(
    model='qwen3.6-plus',
    prompt=prompt,
    use_context_cache=True
)
```

2.5.3 成本监控与分析

建议使用阿里云的成本中心（Cost Center）进行精细化管理：

步骤一：设置预算告警 - 按月/季度设置预算上限 - 达到 80% 时发送预警 - 达到 100% 时自动暂停非关键服务

步骤二：按项目分摊成本 - 为每个业务线创建独立的 API Key - 通过标签（Tag）标记不同用途 - 定期生成成本分析报告

步骤三：识别异常消耗 - 监控单日费用突增 - 排查死循环调用或恶意攻击 - 建立异常消耗的应急响应流程

本章小结

本章系统介绍了阿里云百炼平台的整体架构与核心技术，重点讲解了以下内容：

1. **平台架构**：百炼平台采用四层架构设计（基础设施层、模型层、服务层、应用层），支持中国内地

2. **资源调度**：GPU 池化技术通过 token 级调度将 GPU 利用率从 30% 提升至 75%+，结合弹性伸缩和上下文缓存机制，实现性能与成本的最优平衡。
3. **模型家族**：Qwen3.5-Omni 在全模态能力上达到行业领先水平，Qwen3.6-Plus 在编程任务上媲美国际顶尖模型且价格仅为 1/15，为企业提供了多样化的选择。
4. **SLA 保障**：通过多地域冗余、多模型降级、本地缓存兜底等策略，可实现 99.99% 的高可用性，满足关键业务系统的稳定性要求。
5. **成本优化**：合理选择计费模式、优化 Prompt 设计、利用 Batch 调用和上下文缓存，可将总体成本 50-70%。

作为技术决策者，在选择大模型基础设施时，需要综合考虑性能、成本、合规、生态等多个维度。阿里

延伸阅读

1. **阿里云官方文档**
   - 百炼平台产品文档
   - 通义千问模型家族
   - GPU 池化技术白皮书
2. **学术论文**
   - Aegaeon: Token-Level GPU Scheduling for Multi-Model Inference (SOSP 2025)
   - Qwen Technical Report (arXiv 2026)
3. **行业报告**
   - 《2026 中国大模型基础设施市场研究报告》- IDC
   - 《企业级 AI 应用成本优化指南》- 阿里云研究院
4. **实践案例**
   - 某电商平台双 11 大模型服务保障方案
   - 某金融机构 RAG 系统架构设计
   - 某制造企业智能客服成本优化实践

下一章预告：第 3 章将深入探讨大模型 API 开发与集成，包括 RESTful API 设计规范、SDK 使用指南、认证鉴权机制、限流熔断策略等内容，并提供完整的代码示例。

第 3 章 大模型 API 开发与集成

本章导读

API 是大模型能力对外输出的核心接口。对于技术决策者而言，理解 API 设计规范、掌握 SDK 使用方法、建立完善的认证鉴权与限流熔断机制，是构建稳定可靠的大模型应

RESTful API 设计原则出发，深入讲解阿里云百炼平台的 API 调用流程、多语言 SDK 使用技巧、安全防护策略以及性能优化方法。

核心议题： - RESTful API 设计规范与大模型场景适配 - 阿里云百炼 API 的认证鉴权机制 - Python/Java/Node.js 多语言 SDK 实战 - 限流、熔断、重试策略的工程实现 - API 版本管理与向后兼容实践

3.1 RESTful API 设计规范

3.1.1 大模型 API 的特殊性

传统的 RESTful API 设计遵循”资源导向”原则，将系统抽象为一系列资源（Resource），通过 HTTP 动词（GET、POST、PUT、DELETE）对资源进行操作。然而，大模型 API 具有其特殊性：

特点一：无状态会话 vs 有状态对话 - 传统 API：每次请求独立，服务器不保存客户端状态 - 大模型 API：多轮对话需要维护上下文历史，本质是有状态的

特点二：确定性响应 vs 概率性生成 - 传统 API：相同输入必然产生相同输出 - 大模型 API：即使 temperature=0，仍可能存在微小差异

特点三：低延迟要求 vs 高计算密度 - 传统 API：通常在毫秒级返回 - 大模型 API：首 token 延迟 100-500ms，完整响应可能数秒

基于这些特点，大模型 API 的设计需要在标准 RESTful 规范基础上做出调整。

3.1.2 端点（Endpoint）设计

阿里云百炼平台采用以下端点设计规范：

```
# URL
https://dashscope.aliyuncs.com/api/v1

#
POST /services/aigc/text-generation/generation

#
POST /services/aigc/multimodal-generation/generation

# embeddings
POST /services/embeddings/text-embedding/text-embedding

#
POST /services/audio/tts/synthesis

#
POST /services/audio/asr/transcription
```

设计要点：

1. **版本化路径**：`/api/v1/` 明确标识 API 版本，便于后续迭代
2. **服务分类**：`/services/aigc/`、`/services/embeddings/` 按能力类型分组
3. **动词语义**：使用具体动作（generation、synthesis）而非泛化的 `generate`
4. **复数形式**：资源名称统一使用复数（如 `/embeddings`）

3.1.3 请求体（Request Body）结构

标准的文本生成请求体结构如下：

```
{
  "model": "qwen3.6-plus",
  "input": {
    "messages": [
      {
        "role": "system",
        "content": "       "
      },
      {
        "role": "user",
        "content": "  Python       "
      }
    ]
  },
  "parameters": {
    "temperature": 0.7,
    "top_p": 0.9,
    "max_tokens": 1024,
    "stream": false,
    "stop": ["\n\n"],
    "seed": 42
  }
}
```

参数详解：

参数	类型	默认值	说明
`model`	string	必填	模型名称，如 `qwen3.6-plus`
`messages`	array	必填	对话历史，包含 system/user/assistant 角色

参数	类型	默认值	说明
`temperature`	float	0.7	控制随机性，0 为确定性，2 为高度随机
`top_p`	float	0.9	核采样参数，控制词汇选择范围
`max_tokens`	integer	模型上限	限制最大输出长度
`stream`	boolean	false	是否启用流式输出
`stop`	array	null	自定义停止词序列
`seed`	integer	null	随机种子，用于结果复现

3.1.4 响应体（Response Body）结构

标准响应体结构：

```
{
  "request_id": "8f3a2c1b-5d6e-4f7a-8b9c-0d1e2f3a4b5c",
  "code": "",
  "message": "",
  "output": {
    "text": "def quicksort(arr):...",
    "finish_reason": "stop"
  },
  "usage": {
    "input_tokens": 45,
    "output_tokens": 89,
    "total_tokens": 134
  }
}
```

字段说明：

- `request_id`：唯一请求标识，用于问题排查和日志追踪
- `code`：错误码，空字符串表示成功
- `message`：错误描述，成功时为空
- `output.text`：生成的文本内容
- `output.finish_reason`：结束原因（`stop` 正常停止 / `length` 达到 max_tokens / `error` 出错）
- `usage`：Token 用量统计，用于计费和对账

3.1.5 错误处理规范

统一的错误响应格式：

```
{
  "request_id": "8f3a2c1b-5d6e-4f7a-8b9c-0d1e2f3a4b5c",
```

```
  "code": "InvalidParameter",
  "message": "The parameter 'temperature' is out of range [0, 2].",
  "http_status_code": 400
}
```

常见错误码：

错误码	HTTP 状态码	说明	解决方案
`InvalidParameter`	400	参数格式错误或超出范围	检查参数类型和取值范围
`InvalidApiKey`	401	API Key 无效或已过期	重新获取 API Key
`QuotaExhausted`	403	配额已用尽	购买更多额度或等待下月重置
`ModelNotFound`	404	指定的模型不存在	确认模型名称正确
`RateLimitExceeded`	429	请求频率超限	降低调用频率或申请提升配额
`InternalError`	500	服务端内部错误	稍后重试，如持续出现需提交工单

3.2 认证鉴权机制

3.2.1 API Key 管理

阿里云百炼平台采用 API Key 作为主要的认证方式。

获取 API Key 的步骤：

1. 登录阿里云控制台 (https://console.aliyun.com)
2. 进入”访问控制 RAM” → “身份管理” → “用户”
3. 创建 RAM 用户（建议为每个应用创建独立用户）
4. 为用户授予”百炼平台只读权限”或”百炼平台管理权限”
5. 创建 AccessKey（包括 AccessKeyId 和 AccessKeySecret）

安全最佳实践：

✅ **推荐做法：** - 为每个环境（开发/测试/生产）创建独立的 API Key - 定期轮换 API Key（建议每季度一次） - 使用环境变量或密钥管理服务存储 API Key - 在代码中绝不硬编码 API Key - 启用 API Key 使用监控和异常告警

❌ **禁止做法：** - 将 API Key 提交到 Git 仓库 - 在前端代码中暴露 API Key - 多人共享同一个 API Key - 长期不更换 API Key

3.2.2 签名认证流程

对于需要更高安全性的场景，百炼平台支持基于 HMAC-SHA256 的签名认证。签名生成需要使用 AccessKey Secret 对请求内容进行加密，确保请求的完整性和真实性。

3.2.3 STS 临时凭证

对于移动端或前端应用，建议使用STS（Security Token Service）临时凭证，避免长期凭证泄露风险。S
临时凭证的有效期通常为 1-12 小时，过期自动失效，大幅降低了安全风险。

3.3 多语言 SDK 实战

3.3.1 Python SDK

安装：

```
pip install dashscope
```

基础调用示例：

```python
import os
from http import HTTPStatus
import dashscope
from dashscope import Generation

#      API Key
dashscope.api_key = os.getenv("DASHSCOPE_API_KEY")

response = Generation.call(
    model='qwen3.6-plus',
    messages=[
        {'role': 'system', 'content': '       '},
        {'role': 'user', 'content': '  Python       '}
    ],
    temperature=0.7,
    max_tokens=1024
)

if response.status_code == HTTPStatus.OK:
    print(f"  {response.output.choices[0].message.content}")
    print(f"Token   {response.usage}")
else:
    print(f"   {response.code} - {response.message}")
```

流式输出示例：

```python
responses = Generation.call(
    model='qwen3.6-plus',
    messages=[{'role': 'user', 'content': '        '}],
    stream=True,
    incremental_output=True
```

```
)

for response in responses:
    if response.status_code == HTTPStatus.OK:
        delta = response.output.choices[0].message.content
        print(delta, end='', flush=True)
```

3.3.2 Java SDK

Maven 依赖：

```
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>dashscope-sdk-java</artifactId>
    <version>2.15.0</version>
</dependency>
```

基础调用示例：

```
import com.alibaba.dashscope.aigc.generation.Generation;
import com.alibaba.dashscope.aigc.generation.GenerationParam;
import com.alibaba.dashscope.aigc.generation.GenerationResult;
import com.alibaba.dashscope.common.Message;
import com.alibaba.dashscope.common.Role;

Generation gen = new Generation();

Message userMsg = Message.builder()
    .role(Role.USER.getValue())
    .content("  Java     ")
    .build();

GenerationParam param = GenerationParam.builder()
    .apiKey("sk-xxxxxxxx")
    .model("qwen3.6-plus")
    .addMessage(userMsg)
    .temperature(0.7)
    .maxTokens(1024)
    .build();

GenerationResult result = gen.call(param);
System.out.println("  " + result.getOutput().getChoices().get(0).getMessage().getContent
```

3.3.3 Node.js SDK

安装：

```
npm install @alicloud/dashscope
```

基础调用示例：

```javascript
const DashScope = require('@alicloud/dashscope');

const client = new DashScope({
  accessKeyId: process.env.DASHSCOPE_API_KEY,
});

async function chat() {
  const response = await client.invoke('qwen3.6-plus', {
    messages: [
      { role: 'user', content: '  JavaScript     ' }
    ],
    parameters: {
      temperature: 0.7,
      max_tokens: 1024
    }
  });

  console.log(' :', response.output.choices[0].message.content);
}

chat();
```

3.3.4 HTTP 原生调用

对于不支持 SDK 的语言或环境，可以直接使用 HTTP 请求：

```python
import requests

url = "https://dashscope.aliyuncs.com/api/v1/services/aigc/text-generation/generation"
headers = {
    "Authorization": "Bearer sk-xxxxxxxx",
    "Content-Type": "application/json"
}
payload = {
    "model": "qwen3.6-plus",
    "input": {
        "messages": [{"role": "user", "content": " "}]
    }
}

response = requests.post(url, headers=headers, json=payload)
result = response.json()
```

```
print(f"  {result['output']['text']}")
```

3.4 限流熔断策略

3.4.1 限流（Rate Limiting）

百炼平台的限流规则：

用户等级	QPS 限制	RPM 限制	并发连接数
免费用户	2 QPS	60 RPM	5
按量付费	10 QPS	300 RPM	20
企业套餐	50 QPS	1500 RPM	100

客户端限流实现（令牌桶算法）：

```
import time
from threading import Lock

class RateLimiter:
    def __init__(self, rate, capacity):
        self.rate = rate  #
        self.capacity = capacity  #
        self.tokens = capacity
        self.last_update = time.time()
        self.lock = Lock()

    def acquire(self):
        with self.lock:
            now = time.time()
            elapsed = now - self.last_update
            self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
            self.last_update = now

            if self.tokens >= 1:
                self.tokens -= 1
                return True
            return False

#
limiter = RateLimiter(rate=10, capacity=20)

if limiter.acquire():
```

```
    response = call_api()
else:
    print("    ")
```

3.4.2 熔断（Circuit Breaker）

熔断器用于在服务连续失败时快速失败，避免雪崩效应：

```
from enum import Enum

class CircuitState(Enum):
    CLOSED = "closed"
    OPEN = "open"
    HALF_OPEN = "half_open"

class CircuitBreaker:
    def __init__(self, failure_threshold=5, recovery_timeout=60):
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.failure_count = 0
        self.last_failure_time = None
        self.state = CircuitState.CLOSED

    def call(self, func, *args, **kwargs):
        if self.state == CircuitState.OPEN:
            if time.time() - self.last_failure_time > self.recovery_timeout:
                self.state = CircuitState.HALF_OPEN
            else:
                raise Exception("    ")

        try:
            result = func(*args, **kwargs)
            self.failure_count = 0
            self.state = CircuitState.CLOSED
            return result
        except Exception as e:
            self.failure_count += 1
            self.last_failure_time = time.time()
            if self.failure_count >= self.failure_threshold:
                self.state = CircuitState.OPEN
            raise e
```

3.4.3 重试（Retry）策略

带指数退避的智能重试：

```python
import time
import random

def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=60.0):
    def decorator(func):
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(max_retries + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last_exception = e
                    if attempt == max_retries:
                        break
                    delay = min(base_delay * (2 ** attempt), max_delay)
                    delay += random.uniform(0, delay * 0.1)  #
                    print(f"  {attempt+1}/{max_retries},   {delay:.2f}s")
                    time.sleep(delay)
            raise last_exception
        return wrapper
    return decorator

@retry_with_backoff(max_retries=3)
def robust_api_call():
    return call_dashscope_api()
```

3.5 API 版本管理

3.5.1 版本号规范

采用语义化版本号（Semantic Versioning）：

```
Major.Minor.Patch

- Major:    API
- Minor:
- Patch:
```

3.5.2 版本过渡策略

并行运行期： - 新旧版本同时可用至少 3 个月 - 提供迁移指南和自动化脚本 - 监控旧版本使用情况

废弃通知： - HTTP Response Header 添加 `Deprecation` 字段 - 返回警告信息提示升级 - 定期发送邮件通知

最终下线： - 提前 30 天发送最后通牒 - 保留错误日志便于问题排查 - 提供紧急回滚方案

本章小结

本章系统讲解了大模型 API 开发与集成的核心知识：

1. **RESTful API 设计**：针对大模型场景的特殊性，设计了合理的端点、请求体和响应体结构，规范了错
2. **认证鉴权**：采用 API Key 作为主要认证方式，支持 HMAC-SHA256 签名和 STS 临时凭证，满足不同的安全需求。
3. **多语言 SDK**：提供了 Python、Java、Node.js 等多种语言的 SDK 使用示例，以及 HTTP 原生调用方法。
4. **弹性防护**：通过限流、熔断、重试三重机制，构建了高可用的 API 调用体系，有效应对流量峰值和
5. **版本管理**：采用语义化版本号，制定清晰的版本过渡策略，确保 API 演进的平滑性。

掌握这些技能，技术决策者可以带领团队构建出稳定、安全、高效的大模型应用。

延伸阅读

1. 阿里云百炼 API 参考文档
2. DashScope SDK GitHub
3. 《微服务架构设计模式》- Chris Richardson
4. 《API Design Patterns》- JJ Geewax

下一章预告：第 4 章将深入探讨 RAG 应用架构与实战，包括知识库构建、向量检索、文档切分、检索优 RAG 系统实现案例。

第 4 章 RAG 应用架构与实战

本章导读

检索增强生成（Retrieval-Augmented Generation，RAG）是解决大模型幻觉问题、提升专业知 RAG 系统的整体架构、知识库构建流程、向量检索优化、文档切分策略等关键议题，并提供基于阿 DashVector 的完整实现方案。

核心议题： - RAG 技术原理与适用场景 - 知识库构建全流程（文档采集、清洗、切分、向量化）
- 向量数据库选型与 DashVector 实战 - 检索优化策略（混合检索、重排序、多跳检索）
- 企业级 RAG 系统案例分析

4.1 RAG 技术概述

4.1.1 为什么需要 RAG?

大语言模型虽然强大，但存在三个固有局限：

局限一：知识截止 - 训练数据有明确的时间截止点 - 无法获知训练后的新信息 - 例如：GPT-4 的知识截止到 2024 年 4 月

局限二：私有知识缺失 - 训练数据来自公开互联网 - 不包含企业内部文档、产品手册、客户案例等私有信
- 无法回答涉及公司内部流程的问题

局限三：幻觉问题 - 面对不确定的问题时倾向于”编造”答案 - 在医疗、法律、金融等专业领域风险极高
- 难以追溯答案的信息来源

RAG 技术通过”先检索后生成”的机制，有效解决了上述问题：

```
→      →   Prompt →     →
```

4.1.2 RAG 工作流程

标准的 RAG 工作流程包含两个阶段：

阶段一：离线知识库构建

```
→     →     →     →     →
```

阶段二：在线检索增强

```
→     →     → Top-K   → ( )    →   Prompt →
```

4.1.3 阿里云百炼 RAG 架构

阿里云百炼平台提供了一站式的 RAG 解决方案，核心组件包括：

组件	功能	对应产品
文档处理	PDF/Word/Excel 解析、OCR、文本清洗	百炼数据处理服务
向量化	文本 embedding、多语言支持	text-embedding-v2/v3
向量存储	高维向量索引、相似度检索	DashVector / 阿里云 Milvus 版
检索引擎	语义检索、混合检索、多路召回	百炼检索服务
生成模型	基于检索结果生成答案	Qwen 系列模型

4.2 知识库构建

4.2.1 文档采集

企业知识通常分散在多种载体中：

常见文档类型： - Office 文档：Word、PPT、Excel - PDF 文件：产品手册、技术文档、合同 - 网页内容：官网 FAQ、帮助中心 - 数据库记录：工单系统、CRM、知识库 - 即时通讯：钉钉聊天记录、

采集方式：

```
#
from langchain.document_loaders import (
    PyPDFLoader,
    Docx2txtLoader,
    UnstructuredMarkdownLoader,
    WebBaseLoader
)

# PDF
pdf_loader = PyPDFLoader("product_manual.pdf")
pdf_docs = pdf_loader.load()

# Word
docx_loader = Docx2txtLoader("specifications.docx")
docx_docs = docx_loader.load()

#
web_loader = WebBaseLoader("https://help.example.com/faq")
web_docs = web_loader.load()

#
all_docs = pdf_docs + docx_docs + web_docs
print(f"   {len(all_docs)}    ")
```

4.2.2 数据清洗

原始文档通常包含大量噪声，需要清洗：

常见噪声类型： - 页眉页脚、页码 - 广告、导航菜单 - 特殊字符、乱码 - 无意义的短文本 - 重复内容

清洗策略：

```
import re

def clean_text(text):
    #
```

```
    text = re.sub(r'\s+', ' ', text).strip()

    #
    text = re.sub(r'^ \d+ .*', '', text, flags=re.MULTILINE)

    #
    text = re.sub(r'[^\w\s\u4e00-\u9fff.,!?;:()""\'\-]', '', text)

    #       10
    if len(text) < 10:
        return None

    return text

#
cleaned_docs = []
for doc in all_docs:
    cleaned_text = clean_text(doc.page_content)
    if cleaned_text:
        doc.page_content = cleaned_text
        cleaned_docs.append(doc)

print(f"    {len(cleaned_docs)}    ")
```

4.2.3 文档切分

合理的切分策略直接影响检索效果：

切分原则： - 保持语义完整性（不要在句子中间切断） - 控制片段长度（通常 200-500 字） - 设置重叠区域（避免上下文丢失） - 考虑文档结构（按章节、段落切分）

常用切分方法：

```
from langchain.text_splitter import (
    CharacterTextSplitter,
    RecursiveCharacterTextSplitter,
    MarkdownHeaderTextSplitter
)

#   1
splitter1 = CharacterTextSplitter(
    separator="\n",
    chunk_size=300,
    chunk_overlap=50
)
```

```
#   2
splitter2 = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", " ", " ", " ", " ", ""],
    chunk_size=300,
    chunk_overlap=50,
    length_function=len
)

#   3  Markdown
splitter3 = MarkdownHeaderTextSplitter(
    headers_to_split_on=[
        ("#", "Header 1"),
        ("##", "Header 2"),
        ("###", "Header 3")
    ]
)

#
chunks = splitter2.split_documents(cleaned_docs)
print(f"   {len(chunks)}   ")
print(f"   {sum(len(c.page_content) for c in chunks) / len(chunks):.0f}  ")
```

4.2.4 向量化（Embedding）

将文本转换为高维向量：

阿里云 Embedding 模型：

模型	维度	最大输入	特点
text-embedding-v2	1536	512 tokens	通用场景，性价比高
text-embedding-v3	2048	8192 tokens	高精度，支持多语言

向量化代码示例：

```
import dashscope
from dashscope import TextEmbedding

dashscope.api_key = "sk-xxx"

def get_embedding(text):
    response = TextEmbedding.call(
        model='text-embedding-v3',
        input=text
    )
```

```
    if response.status_code == 200:
        return response.output['embeddings'][0]['embedding']
    else:
        raise Exception(f"Embedding   {response.message}")

#
for chunk in chunks:
    chunk.vector = get_embedding(chunk.page_content)
```

4.2.5 向量存储

使用向量数据库存储和索引：

DashVector 快速入门：

```
from dashvector import Client, Doc

#
client = Client(api_key="sk-xxx", endpoint="https://xxx.dashvector.aliyuncs.com")

#    Collection
client.create(
    name='knowledge_base',
    dimension=2048,  #   embedding
    metric='cosine'  #
)

#
docs = [
    Doc(
        id=f"doc_{i}",
        vector=chunk.vector,
        fields={
            'content': chunk.page_content,
            'source': chunk.metadata.get('source', ''),
            'page': chunk.metadata.get('page', 0)
        }
    )
    for i, chunk in enumerate(chunks)
]

collection = client.get('knowledge_base')
collection.upsert(docs)
print(f"   {len(docs)}   ")
```

4.3 检索优化策略

4.3.1 相似度检索

基础向量检索：

```
def search(query, top_k=5):
    #
    query_vector = get_embedding(query)

    #
    collection = client.get('knowledge_base')
    result = collection.query(
        vector=query_vector,
        topk=top_k,
        include_fields=['content', 'source']
    )

    return [doc.fields['content'] for doc in result.docs]

#
contexts = search("    ", top_k=5)
```

4.3.2 混合检索（Hybrid Search）

结合语义检索和关键词检索的优势：

```
from elasticsearch import Elasticsearch

# ES
es = Elasticsearch("http://localhost:9200")

def hybrid_search(query, top_k=5):
    # 1.
    query_vector = get_embedding(query)
    vector_results = collection.query(
        vector=query_vector,
        topk=top_k * 2
    )

    # 2.
    es_results = es.search(
        index="knowledge_base",
        query={
            "multi_match": {
                "query": query,
```

```
                "fields": ["content", "title"]
            }
        },
        size=top_k * 2
    )

    # 3.    RRF
    fused_scores = {}
    for rank, doc in enumerate(vector_results.docs):
        doc_id = doc.id
        fused_scores[doc_id] = fused_scores.get(doc_id, 0) + 1 / (rank + 1)

    for rank, hit in enumerate(es_results['hits']['hits']):
        doc_id = hit['_id']
        fused_scores[doc_id] = fused_scores.get(doc_id, 0) + 1 / (rank + 1)

    # 4.   Top-K
    sorted_docs = sorted(fused_scores.items(), key=lambda x: x[1], reverse=True)[:top_k]

    return sorted_docs
```

4.3.3 重排序（Re-ranking）

对初筛结果进行精细排序：

```
from dashscope import TextRank

def rerank(query, documents, top_k=3):
    """
      Cross-Encoder
    """
    response = TextRank.call(
        model='gte-rerank',
        query=query,
        docs=[doc for doc in documents]
    )

    #
    ranked_indices = [item['index'] for item in response.output['results']]

    #   Top-K
    return [documents[i] for i in ranked_indices[:top_k]]

#
initial_results = hybrid_search(query, top_k=10)
```

```
final_results = rerank(query, initial_results, top_k=5)
```

4.3.4 多跳检索（Multi-hop Retrieval）

对于复杂问题，可能需要多次检索：

```
def multi_hop_search(question, max_hops=3):
    contexts = []
    current_question = question

    for hop in range(max_hops):
        #   1
        results = search(current_question, top_k=3)
        contexts.extend(results)

        #
        synthesis_prompt = f"""


          {question}
           {''.join(results)}

              " "
        """

        llm_response = call_qwen(synthesis_prompt)

        if " " in llm_response:
            break
        else:
            #
            next_query_prompt = f"""
                {question}

            """
            current_question = call_qwen(next_query_prompt)

    return contexts
```

4.4 完整 RAG 系统实现

4.4.1 系统架构

API Gateway
(Web/App) (Flask)

API
DashVector MySQL/RDS Qwen

4.4.2 核心代码

```
from flask import Flask, request, jsonify
from dashscope import Generation
import dashvector

app = Flask(__name__)

#
dv_client = dashvector.Client(api_key="xxx", endpoint="xxx")
collection = dv_client.get('knowledge_base')

@app.route('/api/rag/query', methods=['POST'])
def rag_query():
    data = request.json
    question = data.get('question', '')

    if not question:
        return jsonify({'error': '    '}), 400

    # 1.
    query_vector = get_embedding(question)

    # 2.    Top-5
    result = collection.query(
        vector=query_vector,
        topk=5,
        include_fields=['content', 'source']
    )
```

```
    contexts = [doc.fields['content'] for doc in result.docs]
    sources = [doc.fields['source'] for doc in result.docs]

    # 3.     Prompt
    prompt = build_rag_prompt(question, contexts)

    # 4.
    response = Generation.call(
        model='qwen3.6-plus',
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0.7
    )

    answer = response.output.choices[0].message.content

    # 5.
    return jsonify({
        'answer': answer,
        'contexts': contexts,
        'sources': list(set(sources)),
        'model': 'qwen3.6-plus'
    })

def build_rag_prompt(question, contexts):
    context_text = "\n\n".join([f"[ {i+1}]\n{ctx}" for i, ctx in enumerate(contexts)])

    return f"""


{context_text}

  {question}

  """

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

本章小结

本章深入讲解了 RAG 技术的原理与实战：

1. **RAG 价值**：解决大模型的知识截止、私有知识缺失、幻觉三大问题
2. **知识库构建**：文档采集→清洗→切分→向量化→存储的完整流程
3. **检索优化**：混合检索、重排序、多跳检索等多种提升精度的策略
4. **系统实现**：基于阿里云百炼和 DashVector 的企业级 RAG 架构

RAG 已成为企业构建专业知识问答系统的首选方案，掌握这项技术对于技术决策者至关重要。

延伸阅读

1. 阿里云百炼知识库官方文档
2. DashVector 向量数据库
3. 《Retrieval-Augmented Generation for Large Language Models》- Facebook AI
4. LangChain RAG 最佳实践：https://python.langchain.com/docs/use_cases/retrievers

---

下一章预告：第 5 章将探讨 Agent 应用架构与实战，包括 Agent 的核心组件、规划能力、工具调用、记
Agent 开发框架。

第 5 章 Agent 应用架构与实战

本章导读

AI Agent（智能体）是大模型应用的高级形态，具备自主规划、工具调用、记忆保持和目标导向等
Agent 的核心架构、规划算法、工具集成机制、记忆系统设计，并提供基于阿里云百炼平台的
Agent 开发框架和实战案例。

核心议题： - Agent 的定义与核心能力 - ReAct、Plan-and-Solve 等规划算法 - 工具调用（Function Calling）机制 - 短期记忆与长期记忆设计 - 多 Agent 协作模式 - 百炼平台 Agent 开发实战

5.1 Agent 核心概念

5.1.1 什么是 AI Agent?

AI Agent 是具备环境感知、决策推理与行动执行能力的自治系统。与传统被动响应的 AI 不同，Agent 能够：

特性	传统 AI	AI Agent
响应模式	被动问答	主动规划
任务范围	单一任务	多步骤复杂任务
工具使用	无	可调用外部 API/工具
记忆能力	无状态	短期 + 长期记忆
目标导向	无	有明确目标并持续追踪

典型应用场景： - 智能客服：自动处理退款、改签等复杂流程 - 代码助手：理解需求→编写代码→运行测
bug - 数据分析：获取数据→清洗→分析→生成报告 - 个人助理：安排行程、预订餐厅、发送邮件

5.1.2 Agent 四层架构

```
                (Planning)



                (Tool Use)

    API

                (Memory)

            episodic     semantic

                (Perception)
```

5.2 规划能力（Planning）

5.2.1 任务分解（Task Decomposition）

将复杂任务拆解为可执行的子任务：

示例：数据分析任务

" "

1. Q3
2.
3.
4.
5.
6.

代码实现：

```
from dashscope import Generation

def decompose_task(task_description):
    """

    """
    prompt = f"""


  {task_description}


1. [  1]
2. [  2]
...
"""

    response = Generation.call(
        model='qwen3.6-plus',
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0.3  #
    )

    plan_text = response.output.choices[0].message.content

    #
    subtasks = []
    for line in plan_text.split('\n'):
        if line.strip() and line[0].isdigit():
            task = line.split('.', 1)[1].strip().strip('[]')
            subtasks.append(task)

    return subtasks
```

```
#
subtasks = decompose_task("        ")
for i, task in enumerate(subtasks, 1):
    print(f"{i}. {task}")
```

5.2.2 ReAct 算法（Reasoning + Acting）

ReAct 是当前最主流的 Agent 算法，核心思想是交替进行推理和行动：

```
Thought:
Action: query_order_api(user_id="U12345")
Observation: {"order_id": "O789", "status": "shipped", ...}
Thought:
Action: query_logistics_api(order_id="O789")
Observation: {"location": " ", "estimated_delivery": "2026-04-18"}
Thought:
Final Answer:         4   18
```

ReAct 实现框架：

```
class ReActAgent:
    def __init__(self, model='qwen3.6-plus', tools=None):
        self.model = model
        self.tools = tools or {}
        self.max_iterations = 10

    def run(self, question):
        """
          ReAct
        """
        history = []

        for iteration in range(self.max_iterations):
            # 1.   Thought   Action
            prompt = self._build_prompt(question, history)
            response = Generation.call(
                model=self.model,
                messages=[{'role': 'user', 'content': prompt}]
            )
            output = response.output.choices[0].message.content

            # 2.
            thought, action, action_input = self._parse_output(output)

            if not action:
                #   Action
```

```
                return thought

            # 3.   Action
            observation = self._execute_action(action, action_input)

            # 4.
            history.append({
                'thought': thought,
                'action': action,
                'action_input': action_input,
                'observation': observation
            })

        raise Exception("        ")

    def _build_prompt(self, question, history):
        """  ReAct Prompt"""
        tools_desc = "\n".join([
            f"{name}: {desc}" for name, (desc, _) in self.tools.items()
        ])

        prompt = f"""

{tools_desc}

  {question}

"""
        #
        for i, step in enumerate(history):
            prompt += f"""Step {i+1}:
Thought: {step['thought']}
Action: {step['action']}
Action Input: {step['action_input']}
Observation: {step['observation']}

"""

        prompt += f"Step {len(history)+1}:\nThought:"
        return prompt

    def _parse_output(self, output):
        """  Thought/Action"""
        import re
```

```
        thought_match = re.search(r'Thought:\s*(.+?)(?=Action:|$)', output, re.DOTALL)
        action_match = re.search(r'Action:\s*(\w+)', output)
        input_match = re.search(r'Action Input:\s*(.+?)(?=Observation:|$)', output, re.D

        thought = thought_match.group(1).strip() if thought_match else ""
        action = action_match.group(1) if action_match else None
        action_input = input_match.group(1).strip() if input_match else ""

        return thought, action, action_input

    def _execute_action(self, action_name, action_input):
        """    """
        if action_name not in self.tools:
            return f"     {action_name}"

        _, func = self.tools[action_name]
        try:
            #   JSON
            import json
            params = json.loads(action_input)
            result = func(**params)
            return str(result)
        except Exception as e:
            return f"   {str(e)}"

#
def query_order_api(user_id):
    return {"order_id": "O789", "status": "shipped", "amount": 299.00}

def query_logistics_api(order_id):
    return {"location": " ", "estimated_delivery": "2026-04-18"}

tools = {
    "query_order_api": ("     ", query_order_api),
    "query_logistics_api": ("     ", query_logistics_api)
}

#     Agent
agent = ReActAgent(tools=tools)
result = agent.run("       ID: U12345")
print(result)
```

5.2.3 Plan-and-Solve 算法

相比 ReAct，Plan-and-Solve 更强调先制定完整计划再执行：

```
def plan_and_solve(problem, model='qwen3.6-plus'):
    # Phase 1:
    plan_prompt = f"""
      {problem}



    """

    plan_response = Generation.call(
        model=model,
        messages=[{'role': 'user', 'content': plan_prompt}]
    )
    plan = plan_response.output.choices[0].message.content

    # Phase 2:
    solve_prompt = f"""
      {problem}


    {plan}


    """

    solve_response = Generation.call(
        model=model,
        messages=[{'role': 'user', 'content': solve_prompt}]
    )

    return solve_response.output.choices[0].message.content
```

5.3 工具调用（Function Calling）

5.3.1 Function Calling 机制

Qwen3.6-Plus 原生支持 Function Calling，可以精准调用外部工具：

```
from dashscope import Generation
import json
```

```python
#    Schema
tools_schema = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "     ",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "    ' ' ' '"
                    },
                    "date": {
                        "type": "string",
                        "description": "    YYYY-MM-DD    "
                    }
                },
                "required": ["city"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "calculate",
            "description": "    ",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "    '2+2' 'sqrt(16)'"
                    }
                },
                "required": ["expression"]
            }
        }
    }
]

#
def get_weather(city, date=None):
```

```
    #    API
    return f"{city}     25°C"

def calculate(expression):
    #
    allowed_chars = set("0123456789+-*/.() ")
    if all(c in allowed_chars for c in expression):
        return str(eval(expression))
    return "   "

tools_map = {
    "get_weather": get_weather,
    "calculate": calculate
}

#
response = Generation.call(
    model='qwen3.6-plus',
    messages=[
        {'role': 'user', 'content': '          123*456   '}
    ],
    tools=tools_schema,
    tool_choice='auto'  #
)

#
message = response.output.choices[0].message

if message.tool_calls:
    #
    for tool_call in message.tool_calls:
        func_name = tool_call.function.name
        func_args = json.loads(tool_call.function.arguments)

        #
        result = tools_map[func_name](**func_args)
        print(f"  {func_name},   {result}")

        #
        response2 = Generation.call(
            model='qwen3.6-plus',
            messages=[
                {'role': 'user', 'content': message.content},
                {'role': 'assistant', 'content': message.content, 'tool_calls': message.
                {'role': 'tool', 'content': result, 'tool_call_id': tool_call.id}
```

```
            ]
        )
        print(f"   {response2.output.choices[0].message.content}")
else:
    #
    print(message.content)
```

5.3.2 自定义工具封装

将企业现有系统封装为 Agent 工具：

```
class ToolRegistry:
    """   """

    def __init__(self):
        self.tools = {}

    def register(self, name, description, func, param_schema):
        """   """
        self.tools[name] = {
            "schema": {
                "type": "function",
                "function": {
                    "name": name,
                    "description": description,
                    "parameters": param_schema
                }
            },
            "func": func
        }

    def get_all_schemas(self):
        """     Schema"""
        return [t["schema"] for t in self.tools.values()]

    def execute(self, name, **kwargs):
        """  """
        if name not in self.tools:
            raise ValueError(f"   {name}")
        return self.tools[name]["func"](**kwargs)

#      CRM
registry = ToolRegistry()

def create_customer(name, phone, email=None):
```

```
    """    """
    #      CRM API
    customer_id = f"CUST_{hash(name) % 10000}"
    return {"customer_id": customer_id, "status": "created"}

def query_customer(customer_id):
    """    """
    return {"customer_id": customer_id, "name": " ", "level": "VIP"}

#
registry.register(
    name="create_customer",
    description="  CRM      ",
    func=create_customer,
    param_schema={
        "type": "object",
        "properties": {
            "name": {"type": "string", "description": "   "},
            "phone": {"type": "string", "description": "   "},
            "email": {"type": "string", "description": "     "}
        },
        "required": ["name", "phone"]
    }
)

registry.register(
    name="query_customer",
    description="     ",
    func=query_customer,
    param_schema={
        "type": "object",
        "properties": {
            "customer_id": {"type": "string", "description": "  ID"}
        },
        "required": ["customer_id"]
    }
)

# Agent
agent_tools = registry.get_all_schemas()
```

5.4 记忆系统设计

5.4.1 短期记忆（工作记忆）

保存当前对话的上下文：

```
class ShortTermMemory:
    def __init__(self, max_tokens=4000):
        self.messages = []
        self.max_tokens = max_tokens
        self.current_tokens = 0

    def add(self, role, content):
        """  """
        from tiktoken import encoding_for_model
        enc = encoding_for_model("qwen3.6-plus")
        tokens = len(enc.encode(content))

        #
        while self.current_tokens + tokens > self.max_tokens and len(self.messages) > 1:
            old_msg = self.messages.pop(0)
            old_tokens = len(enc.encode(old_msg['content']))
            self.current_tokens -= old_tokens

        self.messages.append({'role': role, 'content': content})
        self.current_tokens += tokens

    def get_messages(self):
        """    """
        return self.messages.copy()

    def clear(self):
        """  """
        self.messages = []
        self.current_tokens = 0

#
memory = ShortTermMemory(max_tokens=4000)
memory.add('user', ' ')
memory.add('assistant', '       ')
memory.add('user', '      ')

conversation = memory.get_messages()
```

5.4.2 长期记忆（向量存储）

将重要信息存入向量数据库，支持跨会话检索：

```
class LongTermMemory:
    def __init__(self, dashvector_client, collection_name='user_memory'):
        self.client = dashvector_client
        self.collection_name = collection_name

        #
        if collection_name not in [c.name for c in client.list()]:
            client.create(name=collection_name, dimension=2048, metric='cosine')

        self.collection = client.get(collection_name)

    def store(self, user_id, content, metadata=None):
        """  """
        from dashscope import TextEmbedding

        #
        embedding_response = TextEmbedding.call(
            model='text-embedding-v3',
            input=content
        )
        vector = embedding_response.output['embeddings'][0]['embedding']

        #
        from dashvector import Doc
        doc = Doc(
            id=f"mem_{user_id}_{int(time.time())}",
            vector=vector,
            fields={
                'user_id': user_id,
                'content': content,
                'timestamp': time.time(),
                **(metadata or {})
            }
        )
        self.collection.upsert([doc])

    def retrieve(self, user_id, query, top_k=3):
        """   """
        from dashscope import TextEmbedding

        #
```

```
        embedding_response = TextEmbedding.call(
            model='text-embedding-v3',
            input=query
        )
        query_vector = embedding_response.output['embeddings'][0]['embedding']

        #
        result = self.collection.query(
            vector=query_vector,
            topk=top_k * 2,  #
            filter=f"user_id='{user_id}'"
        )

        #
        memories = []
        for doc in result.docs:
            recency_score = 1 / (1 + (time.time() - doc.fields['timestamp']) / 86400)
            memories.append({
                'content': doc.fields['content'],
                'score': recency_score
            })

        #    Top-K
        memories.sort(key=lambda x: x['score'], reverse=True)
        return [m['content'] for m in memories[:top_k]]

#
long_memory = LongTermMemory(client=dv_client)

#
long_memory.store(
    user_id="U12345",
    content="       ",
    metadata={'category': 'preference'}
)

#
query = "     "
memories = long_memory.retrieve("U12345", query)
print(memories)  # ["       "]
```

5.4.3 记忆总结机制

定期将对话历史压缩为摘要：

```
def summarize_conversation(messages, model='qwen3.6-plus'):
    """

    """
    conversation_text = "\n".join([
        f"{m['role']}: {m['content']}" for m in messages[-20:]  #    20
    ])

    prompt = f"""

{conversation_text}


1.
2.
3.
4.

        200    """

    response = Generation.call(
        model=model,
        messages=[{'role': 'user', 'content': prompt}]
    )

    return response.output.choices[0].message.content

#       10
if len(memory.messages) % 10 == 0:
    summary = summarize_conversation(memory.messages)
    long_memory.store(user_id, summary, metadata={'type': 'summary'})
```

5.5 多 Agent 协作

5.5.1 角色分工模式

复杂任务可由多个专业 Agent 协作完成：

```
     Agent           Agent           Agent
   (Planner)          (Worker)            (Reviewer)
```

代码示例：

```
class MultiAgentSystem:
    def __init__(self):
        self.planner = ReActAgent(model='qwen3.6-plus')
        self.worker = ReActAgent(model='qwen3.6-plus')
        self.reviewer = ReActAgent(model='qwen3.6-plus')

    def solve(self, task):
        # 1.   Agent
        plan = self.planner.run(f"       {task}")

        # 2.   Agent
        result = self.worker.run(f"  {task}\n  {plan}\n   ")

        # 3.   Agent
        review = self.reviewer.run(f"""
          {task}
           {result}

                     """)

        # 4.        Agent
        if "   " in review:
            revised_result = self.worker.run(f"      {review}")
            return revised_result

        return result

#
system = MultiAgentSystem()
result = system.solve("   Q3             PPT   ")
```

5.5.2 百炼平台 Agent 编排

阿里云百炼提供可视化的 Agent 编排工具：

```
# agent_workflow.yaml
version: "1.0"
name: customer_service_agent

nodes:
```

```yaml
  - id: intent_recognition
    type: llm
    model: qwen3.6-plus
    prompt: "     / / / "
    output_key: intent

  - id: refund_check
    type: condition
    condition: "${intent_recognition.intent} == ' '"
    true_branch: handle_refund
    false_branch: consult_check

  - id: handle_refund
    type: tool
    tool: query_order_api
    params:
      order_id: "${user_input.order_id}"

  - id: consult_check
    type: condition
    condition: "${intent_recognition.intent} == ' '"
    true_branch: knowledge_search
    false_branch: human_handoff

  - id: knowledge_search
    type: rag
    knowledge_base: product_faq
    top_k: 3

  - id: human_handoff
    type: notification
    channel: dingtalk
    recipient: customer_service_team

triggers:
  - event: user_message
    entry_node: intent_recognition
```

本章小结

本章系统讲解了 Agent 应用的架构与实战：

1. **核心架构**：规划层、工具层、记忆层、感知层四层设计

2. **规划算法**：任务分解、ReAct、Plan-and-Solve 等主流方法
3. **工具调用**：Function Calling 机制、自定义工具封装
4. **记忆系统**：短期记忆管理上下文，长期记忆支持跨会话，摘要机制压缩历史
5. **多 Agent 协作**：角色分工、百炼平台可视化编排

Agent 代表了大模型应用的未来方向，从被动问答走向主动执行。技术决策者应提前布局，构建企业的 Agent 开发能力。

延伸阅读

1. ReAct 论文
2. 阿里云百炼 Agent 开发指南
3. LangChain Agent 文档：https://python.langchain.com/docs/modules/agents/
4. 《Building Agentic AI Systems》- Stanford HAI

下一章预告：第 6 章将探讨大模型应用的安全与合规问题，包括内容安全、数据隐私、模型滥用防护、

第 6 章 大模型应用安全与合规

本章导读

安全与合规是大模型企业级落地的前提。本章将系统讲解大模型应用面临的安全风险、内容审核机

核心议题： - 大模型安全风险分类（提示注入、数据泄露、有害内容） - 内容安全审核技术与实践 - 数据隐私保护与脱敏 - 模型滥用检测与防护 - 中国法规合规要求

6.1 安全威胁分类

6.1.1 提示注入攻击（Prompt Injection）

攻击者通过精心设计的输入，诱导模型绕过安全限制。

直接注入示例：

防御策略：

```
def detect_prompt_injection(user_input):
    injection_patterns = [
        r"ignore\s+(previous|all)\s+(instructions|rules)",
        r"bypass\s+(security|restrictions)",
        r"   ",
        r"    "
    ]

    import re
    for pattern in injection_patterns:
        if re.search(pattern, user_input, re.IGNORECASE):
            return True
    return False

if detect_prompt_injection(user_input):
    return "       "
```

6.1.2 数据泄露风险

防护措施：

```
import re

def sanitize_input(text):
    #
    text = re.sub(r'(\d{3})\d{4}(\d{4})', r'\1****\2', text)
    #
    text = re.sub(r'(\d{6})\d{8}(\d{4})', r'\1********\2', text)
    #
    text = re.sub(r'(\w{2})\w+@(\w+\.\w+)', r'\1***@\2', text)
    return text
```

6.1.3 有害内容生成

包括暴力恐怖、仇恨歧视、色情低俗、虚假信息等。可使用阿里云内容安全服务进行检测：

```
from aliyunsdkgreen.request.v20180509 import TextScanRequest

def content_moderation(text):
    client = AcsClient(access_key_id='xxx', access_key_secret='xxx')
    request = TextScanRequest.TextScanRequest()
    request.set_scenes(["antispam"])
    request.set_content(text.encode('utf-8'))

    response = client.do_action_with_exception(request)
```

```
    result = json.loads(response)

    if result['data']['suggestion'] == 'block':
        return False, "  "
    return True, "  "
```

6.2 数据隐私保护

6.2.1 数据分类分级

级别	数据类型	保护要求
L1 - 公开	官网公开信息	无需特殊保护
L2 - 内部	内部文档	访问控制、加密存储
L3 - 敏感	客户信息、财务数据	严格脱敏、审计日志
L4 - 机密	商业机密、核心技术	禁止上传、本地处理

6.2.2 数据脱敏技术

```
from cryptography.fernet import Fernet
import hashlib

class DataMasking:
    def hash_anonymize(self, data):
        """      """
        return hashlib.sha256(data.encode()).hexdigest()[:16]

    def partial_mask(self, data, visible_chars=4):
        """  """
        if len(data) <= visible_chars:
            return '*' * len(data)
        return data[:visible_chars] + '*' * (len(data) - visible_chars)

    def generalize(self, data, level='city'):
        """  """
        if level == 'city':
            return re.sub(r'(.{0,2}|.)(.*?)', r'\1', data)
        return data

#
masker = DataMasking()
masked_phone = masker.partial_mask('13812345678', 3)  # 138*****
```

6.3 模型滥用防护

6.3.1 异常使用检测

```
class AbuseDetection:
    def detect_abuse(self, user_id, request):
        anomalies = []

        #
        if self.get_hourly_count(user_id) > 100:
            anomalies.append("    ")

        # Token
        if estimate_tokens(request) > 10000:
            anomalies.append("Token    ")

        #
        if datetime.now().hour < 6 and self.get_hourly_count(user_id) > 50:
            anomalies.append("     ")

        return len(anomalies) == 0, anomalies
```

6.3.2 深度伪造防护

```
def detect_deepfake_risk(prompt):
    risk_keywords = [' ', '   ', ' ', ' ', 'fake']
    risk_score = sum(1 for kw in risk_keywords if kw in prompt.lower())

    if risk_score >= 2:
        return False, "      "
    return True, "  "
```

6.4 合规要求

6.4.1 中国核心法规

1. **《生成式人工智能服务管理暂行办法》**（2023 年 8 月）
   - 内容符合社会主义核心价值观
   - 建立投诉举报机制
   - 对生成内容进行标识

2. **《互联网信息服务算法推荐管理规定》**
    - 算法备案要求
    - 用户知情权保障
3. **《数据安全法》《个人信息保护法》**
    - 数据分类分级管理
    - 个人信息处理告知同意

6.4.2 合规检查清单

```
compliance_checklist = {
    "   ": [
        "     ",
        "       ",
        "      ",
        "     "
    ],
    "   ": [
        "       ",
        "      ",
        "     ",
        "     "
    ],
    "   ": [
        "     ",
        "       ",
        "    "
    ]
}
```

6.5 企业安全治理框架

6.5.1 组织保障

成立人工智能伦理委员会，由高层领导、法务、技术、业务代表组成，下设安全技术组、合规法务组、

6.5.2 制度体系

- **一级制度**：《人工智能应用管理办法》
- **二级制度**：《内容安全管理细则》《数据隐私保护细则》《应急响应预案》
- **三级操作**：《敏感词库维护 SOP》《数据脱敏操作指南》《安全事件处置流程》

6.5.3 技术防护体系

```
  |       |
  |       |
  |       |
    |       |
```

本章小结

本章系统讲解了大模型应用的安全与合规：

1. **安全威胁**：提示注入、数据泄露、有害内容是三大主要风险
2. **内容审核**：建立关键词过滤、机器学习模型、人工审核三层体系
3. **数据隐私**：实施分类分级管理，采用脱敏、加密、差分隐私等技术
4. **滥用防护**：检测异常使用模式，防范深度伪造等恶意应用
5. **合规要求**：遵循生成式 AI 管理办法、算法推荐规定、数据安全法等法规

安全是企业发展的大模型应用的生命线。技术决策者必须将安全合规纳入产品设计的第一天，而非事后

延伸阅读

1. 生成式人工智能服务管理暂行办法
2. 阿里云内容安全服务
3. 《AI Safety Engineering》- Anthropic
4. OWASP Top 10 for LLM Applications

下一章预告：第 7 章将探讨大模型应用性能优化，包括推理加速、缓存策略、批处理优化、模型压缩等

第 7 章 大模型应用性能优化

本章导读

性能是大模型应用用户体验的关键指标。本章将深入讲解推理加速、缓存策略、批处理优化、模型

核心议题： - 推理延迟分析与优化 - 多级缓存策略设计 - 批处理（Batching）优化
- 模型量化与蒸馏 - 流式输出与增量渲染 - 性能监控与调优

7.1 推理延迟分析

7.1.1 延迟构成

大模型推理延迟主要由以下部分组成：

```
   =     +     + Prefill + Decoding

- Prefill      Prompt     token
- Decoding        token
```

典型延迟分布（Qwen3.6-Plus，1000 输入 +500 输出）：

阶段	耗时	占比
网络传输	20ms	5%
排队等待	50ms	12%
Prefill	100ms	24%
Decoding	250ms	59%
总计	**420ms**	**100%**

7.1.2 首 Token 延迟优化

首 Token 延迟（Time to First Token, TTFT）直接影响用户感知：

```python
#
def stream_response(question):
    responses = Generation.call(
        model='qwen3.6-plus',
        messages=[{'role': 'user', 'content': question}],
        stream=True
    )

    first_token_time = None
    for response in responses:
        if first_token_time is None:
            first_token_time = time.time()
            print(f"  Token   {first_token_time - start_time:.2f}s")

        delta = response.output.choices[0].message.content
        print(delta, end='', flush=True)
```

优化策略： - 启用流式输出，边生成边返回 - 优化网络链路，使用就近接入点 - 减少 Prefill 阶段的计算量（缩短 Prompt）

7.2 多级缓存策略

7.2.1 响应缓存

对于重复或相似问题，直接返回缓存结果：

```
import hashlib
import redis
from datetime import timedelta

class ResponseCache:
    def __init__(self, redis_client):
        self.redis = redis_client
        self.ttl = timedelta(hours=24)

    def _get_cache_key(self, question):
        """   Key"""
        return f"qa_cache:{hashlib.md5(question.encode()).hexdigest()}"

    def get(self, question):
        """   """
        key = self._get_cache_key(question)
        cached = self.redis.get(key)
        if cached:
            return json.loads(cached)
        return None

    def set(self, question, answer):
        """   """
        key = self._get_cache_key(question)
        self.redis.setex(
            key,
            self.ttl,
            json.dumps(answer, ensure_ascii=False)
        )

#
cache = ResponseCache(redis.Redis())

def cached_qa(question):
    #
    cached_answer = cache.get(question)
    if cached_answer:
        return cached_answer

    #       API
```

```
    answer = call_llm_api(question)

    #
    cache.set(question, answer)

    return answer
```

7.2.2 语义缓存

使用向量相似度匹配相似问题：

```
class SemanticCache:
    def __init__(self, dashvector_client, threshold=0.9):
        self.client = dashvector_client
        self.collection = client.get('qa_cache')
        self.threshold = threshold  #

    def search(self, question):
        """     """
        #
        query_vector = get_embedding(question)

        #
        result = self.collection.query(
            vector=query_vector,
            topk=1,
            include_fields=['question', 'answer']
        )

        if result.docs:
            doc = result.docs[0]
            similarity = doc.score

            if similarity >= self.threshold:
                return doc.fields['answer']

        return None

    def store(self, question, answer):
        """   """
        vector = get_embedding(question)
        doc = Doc(
            id=f"cache_{int(time.time())}",
            vector=vector,
            fields={
```

```
                'question': question,
                'answer': answer
            }
        )
        self.collection.upsert([doc])

#
#   "    "
#    "    "
```

7.2.3 上下文缓存

利用百炼平台的上下文缓存功能：

```
#      Prompt
response = Generation.call(
    model='qwen3.6-plus',
    messages=[
        {'role': 'system', 'content': LONG_SYSTEM_PROMPT},
        {'role': 'user', 'content': user_question}
    ],
    use_context_cache=True  #
)

#       Prompt
#    50%
```

7.3 批处理优化

7.3.1 请求合并

将多个小请求合并为批量请求：

```
import asyncio
from collections import deque

class BatchProcessor:
    def __init__(self, batch_size=10, max_wait_ms=100):
        self.batch_size = batch_size
        self.max_wait_ms = max_wait_ms
        self.queue = deque()
        self.lock = asyncio.Lock()

    async def submit(self, request):
```

```
        """   """
        future = asyncio.Future()
        self.queue.append((request, future))

        #
        if len(self.queue) >= self.batch_size:
            await self.process_batch()

        return await future

    async def process_batch(self):
        """    """
        async with self.lock:
            if not self.queue:
                return

            #
            batch = list(self.queue)[:self.batch_size]
            del self.queue[:self.batch_size]

        requests = [r for r, _ in batch]
        futures = [f for _, f in batch]

        #    API
        results = await self.batch_api_call(requests)

        #
        for future, result in zip(futures, results):
            future.set_result(result)

    async def batch_api_call(self, requests):
        """    API"""
        #   DashScope   batch
        response = Generation.batch_call(
            model='qwen3.6-plus',
            requests=requests
        )
        return response

#
processor = BatchProcessor(batch_size=10)

async def handle_user_question(question):
    result = await processor.submit({'messages': [{'role': 'user', 'content': question}]
    return result
```

7.3.2 动态批处理

根据负载动态调整批大小：

```
class DynamicBatcher:
    def __init__(self):
        self.current_batch_size = 10
        self.latency_history = []

    def adjust_batch_size(self, avg_latency):
        """      """
        target_latency = 200  #    200ms

        if avg_latency > target_latency * 1.5:
            #
            self.current_batch_size = max(1, self.current_batch_size - 2)
        elif avg_latency < target_latency * 0.7:
            #
            self.current_batch_size = min(50, self.current_batch_size + 2)

        return self.current_batch_size
```

7.4 模型压缩

7.4.1 量化（Quantization）

将模型权重从 FP32 转换为 INT8/INT4：

```
   (FP32):     4
INT8       1       75%
INT4       0.5       87.5%
```

量化效果对比：

量化精度	模型体积	推理速度	精度损失
FP32	14GB	1x	0%
INT8	3.5GB	2-3x	<1%
INT4	1.75GB	3-4x	1-3%

7.4.2 知识蒸馏（Knowledge Distillation）

用大模型（教师）训练小模型（学生）：

```
#
def distill_loss(student_output, teacher_output, temperature=4.0):
    """

    """
    #
    student_soft = softmax(student_output / temperature)
    teacher_soft = softmax(teacher_output / temperature)

    # KL
    kd_loss = kl_divergence(student_soft, teacher_soft)

    #
    ce_loss = cross_entropy(student_output, true_labels)

    return kd_loss + ce_loss

#          90%+
#      3-5
```

7.4.3 阿里云百炼模型优化

```
#
response = Generation.call(
    model='qwen3.6-plus',
    messages=messages,
    optimization_config={
        'quantization': 'int8',  # INT8
        'use_cache': True,       #   KV Cache
        'max_batch_size': 32     #
    }
)
```

7.5 流式输出与增量渲染

7.5.1 服务端流式

```
@app.route('/api/chat/stream')
def chat_stream():
    def generate():
        responses = Generation.call(
            model='qwen3.6-plus',
            messages=[{'role': 'user', 'content': request.args.get('q')}],
```

```
            stream=True,
            incremental_output=True
        )

        for response in responses:
            delta = response.output.choices[0].message.content
            yield f"data: {json.dumps({'delta': delta})}\n\n"

    return Response(generate(), mimetype='text/event-stream')
```

7.5.2 前端增量渲染

```
//   SSE
const eventSource = new EventSource('/api/chat/stream?q=' + encodeURIComponent(question)

let fullAnswer = '';
eventSource.onmessage = (event) => {
    const data = JSON.parse(event.data);
    fullAnswer += data.delta;

    //    UI
    document.getElementById('answer').innerHTML = marked.parse(fullAnswer);

    //
    window.scrollTo(0, document.body.scrollHeight);
};
```

7.6 性能监控

7.6.1 关键指标

```
metrics = {
    'latency_p50': 150,    #
    'latency_p95': 300,    # P95
    'latency_p99': 500,    # P99
    'throughput_qps': 50,  #
    'token_per_second': 1000,  #    token
    'cache_hit_rate': 0.35,    #
    'error_rate': 0.001        #
}
```

7.6.2 Prometheus 监控

```python
from prometheus_client import Counter, Histogram, start_http_server

#
REQUEST_COUNT = Counter('llm_requests_total', 'Total LLM requests')
REQUEST_LATENCY = Histogram('llm_request_latency_seconds', 'LLM request latency')
TOKEN_COUNT = Counter('llm_tokens_total', 'Total tokens', ['type'])

#
@REQUEST_LATENCY.time()
def call_llm_with_metrics(question):
    REQUEST_COUNT.inc()
    response = call_llm(question)
    TOKEN_COUNT.labels(type='input').inc(response.usage.input_tokens)
    TOKEN_COUNT.labels(type='output').inc(response.usage.output_tokens)
    return response

#
start_http_server(8000)
```

7.6.3 性能看板

使用 Grafana 搭建可视化看板：

```yaml
# dashboard.yaml
dashboard:
  title: "     "
  panels:
    - title: "    "
      type: "heatmap"
      query: "histogram_quantile(0.95, rate(llm_request_latency_seconds_bucket[5m]))"

    - title: "QPS  "
      type: "graph"
      query: "rate(llm_requests_total[1m])"

    - title: "   "
      type: "gauge"
      query: "rate(cache_hits_total[5m]) / rate(cache_requests_total[5m])"

    - title: "Token   "
      type: "graph"
      query: "rate(llm_tokens_total{type='output'}[1m])"
```

本章小结

本章系统讲解了大模型应用的性能优化：

1. **延迟分析**：理解 Prefill 和 Decoding 阶段的延迟构成，针对性优化
2. **多级缓存**：响应缓存、语义缓存、上下文缓存三层体系，可命中 30-50% 的请求
3. **批处理优化**：请求合并、动态批大小，提升吞吐量
4. **模型压缩**：量化、蒸馏等技术，在精度损失<3% 的情况下提速 3-4 倍
5. **流式输出**：服务端流式 + 前端增量渲染，大幅降低用户感知延迟
6. **性能监控**：建立完善的指标体系和可视化看板

性能优化是一个持续的过程。技术决策者应建立性能基线，定期评估优化效果，在成本、质量、速度之

延伸阅读

1. 阿里云百炼性能优化指南
2. 《Efficient NLP》- Hugging Face
3. vLLM 项目：https://github.com/vllm-project/vllm
4. ONNX Runtime 文档：https://onnxruntime.ai/

下一章预告：第 8 章将探讨大模型应用部署与运维，包括容器化部署、CI/CD 流程、灰度发布、监控告

第 8 章 大模型应用部署与运维

本章导读

从开发环境到生产环境的部署与运维是大模型应用落地的最后一公里。本章将讲解容器化部署、C

核心议题： - Docker 容器化部署 - Kubernetes 集群编排 - CI/CD自动化流程 - 灰度发布与回滚 - 监控告警体系 - 故障排查与应急响应

8.1 容器化部署

8.1.1 Dockerfile 编写

```
#
FROM python:3.11-slim

#
```

```dockerfile
WORKDIR /app

#
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

#
COPY . .

#
ENV DASHSCOPE_API_KEY=${DASHSCOPE_API_KEY}
ENV LOG_LEVEL=INFO

#
EXPOSE 8000

#
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

#
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.1.2 Docker Compose 编排

```yaml
version: '3.8'

services:
  #
  llm-app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DASHSCOPE_API_KEY=${DASHSCOPE_API_KEY}
      - REDIS_HOST=redis
      - MYSQL_HOST=mysql
    depends_on:
      - redis
      - mysql
    deploy:
      replicas: 3
      resources:
        limits:
```

```yaml
          cpus: '2'
          memory: 4G
    restart: always

  # Redis
  redis:
    image: redis:7-alpine
    command: redis-server --appendonly yes
    volumes:
      - redis-data:/data
    ports:
      - "6379:6379"

  # MySQL
  mysql:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=llm_app
    volumes:
      - mysql-data:/var/lib/mysql
    ports:
      - "3306:3306"

  # Nginx
  nginx:
    image: nginx:alpine
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    ports:
      - "80:80"
      - "443:443"
    depends_on:
      - llm-app

volumes:
  redis-data:
  mysql-data:
```

8.1.3 Kubernetes 部署

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
```

```yaml
metadata:
  name: llm-app-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: llm-app
  template:
    metadata:
      labels:
        app: llm-app
    spec:
      containers:
      - name: llm-app
        image: registry.cn-beijing.aliyuncs.com/your-namespace/llm-app:v1.0.0
        ports:
        - containerPort: 8000
        env:
        - name: DASHSCOPE_API_KEY
          valueFrom:
            secretKeyRef:
              name: llm-secrets
              key: dashscope-api-key
        - name: REDIS_HOST
          value: "redis-service"
        resources:
          requests:
            memory: "2Gi"
            cpu: "1000m"
          limits:
            memory: "4Gi"
            cpu: "2000m"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 5
```

```yaml
---
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: llm-app-service
spec:
  selector:
    app: llm-app
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8000
  type: LoadBalancer

---
# hpa.yaml (   )
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: llm-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: llm-app-deployment
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
```

8.2 CI/CD自动化流程

8.2.1 GitHub Actions 配置

```yaml
# .github/workflows/deploy.yml
name: Deploy LLM Application

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  #
  test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        pip install pytest pytest-cov

    - name: Run tests
      run: pytest --cov=app tests/

    - name: Upload coverage
      uses: codecov/codecov-action@v3

  #
  build:
    needs: test
    runs-on: ubuntu-latest
    if: github.event_name == 'push'
    steps:
    - uses: actions/checkout@v3

    - name: Login to ACR
```

```yaml
    uses: docker/login-action@v2
    with:
      registry: registry.cn-beijing.aliyuncs.com
      username: ${{ secrets.ACR_USERNAME }}
      password: ${{ secrets.ACR_PASSWORD }}

  - name: Build and push
    uses: docker/build-push-action@v4
    with:
      context: .
      push: true
      tags: |
        registry.cn-beijing.aliyuncs.com/your-namespace/llm-app:${{ github.sha }}
        registry.cn-beijing.aliyuncs.com/your-namespace/llm-app:latest

#
deploy-staging:
  needs: build
  runs-on: ubuntu-latest
  environment: staging
  steps:
  - uses: azure/k8s-set-context@v3
    with:
      kubeconfig: ${{ secrets.K8S_CONFIG_STAGING }}

  - name: Deploy to staging
    run: |
      kubectl set image deployment/llm-app-deployment \
        llm-app=registry.cn-beijing.aliyuncs.com/your-namespace/llm-app:${{ github.sha
      kubectl rollout status deployment/llm-app-deployment

#
deploy-production:
  needs: deploy-staging
  runs-on: ubuntu-latest
  environment: production
  steps:
  - uses: azure/k8s-set-context@v3
    with:
      kubeconfig: ${{ secrets.K8S_CONFIG_PRODUCTION }}

  - name: Deploy to production
    run: |
      kubectl set image deployment/llm-app-deployment \
        llm-app=registry.cn-beijing.aliyuncs.com/your-namespace/llm-app:${{ github.sha
```

```
        kubectl rollout status deployment/llm-app-deployment
```

8.2.2 蓝绿部署

```
#
#!/bin/bash

CURRENT_VERSION=$1
NEW_VERSION=$2
NAMESPACE="llm-prod"

echo "    $CURRENT_VERSION -> $NEW_VERSION"

# 1.
kubectl apply -f deployment-green.yaml -n $NAMESPACE
kubectl set image deployment/llm-app-green llm-app=registry.cn-beijing.aliyuncs.com/your

# 2.
kubectl rollout status deployment/llm-app-green -n $NAMESPACE

# 3.
./run-smoke-tests.sh green

# 4.
kubectl patch service llm-app-service -n $NAMESPACE -p '{"spec":{"selector":{"pod-templa

# 5.   10
sleep 600

# 6.
kubectl delete deployment llm-app-blue -n $NAMESPACE

echo "    "
```

8.3 监控告警体系

8.3.1 日志收集

```
# Fluentd
<match llm.**>
  @type elasticsearch
  host elasticsearch.logging.svc
  port 9200
```

```
  logstash_format true
  logstash_prefix llm-logs
  flush_interval 5s
</match>
```

日志结构化：

```python
import logging
import json

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_entry = {
            'timestamp': self.formatTime(record),
            'level': record.levelname,
            'logger': record.name,
            'message': record.getMessage(),
            'module': record.module,
            'function': record.funcName,
            'line': record.lineno
        }

        #
        if hasattr(record, 'user_id'):
            log_entry['user_id'] = record.user_id
        if hasattr(record, 'request_id'):
            log_entry['request_id'] = record.request_id
        if hasattr(record, 'token_usage'):
            log_entry['token_usage'] = record.token_usage

        return json.dumps(log_entry, ensure_ascii=False)

#
handler = logging.StreamHandler()
handler.setFormatter(JSONFormatter())
logger.addHandler(handler)
```

8.3.2 告警规则

```yaml
# Prometheus AlertManager
groups:
- name: llm-alerts
  rules:
  #
  - alert: HighErrorRate
```

```yaml
    expr: rate(llm_requests_error_total[5m]) / rate(llm_requests_total[5m]) > 0.05
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "       5%"
      description: "    {{ $value | humanizePercentage }}"

  #
  - alert: HighLatency
    expr: histogram_quantile(0.95, rate(llm_request_latency_seconds_bucket[5m])) > 1
    for: 10m
    labels:
      severity: warning
    annotations:
      summary: "P95     1  "
      description: "  P95   {{ $value | humanizeDuration }}"

  # Token
  - alert: TokenQuotaLow
    expr: llm_token_quota_remaining < 100000
    for: 1h
    labels:
      severity: warning
    annotations:
      summary: "Token      10  "
      description: "  Token: {{ $value }}"

  # Pod
  - alert: PodRestartingFrequently
    expr: rate(kube_pod_container_status_restarts_total[1h]) > 0.5
    for: 15m
    labels:
      severity: warning
    annotations:
      summary: "Pod    "
      description: "  1     {{ $value }}"
```

8.3.3 告警通知

```yaml
# AlertManager
receivers:
- name: 'dingtalk'
  webhook_configs:
```

```yaml
  - url: 'http://webhook-dingtalk.monitoring.svc:8060/dingtalk/webhook1/send'
    send_resolved: true

- name: 'email'
  email_configs:
  - to: 'team@example.com'
    from: 'alerts@example.com'
    smarthost: 'smtp.example.com:587'
    auth_username: 'alerts@example.com'
    auth_password: 'xxx'

- name: 'sms'
  webhook_configs:
  - url: 'http://sms-gateway.monitoring.svc/send'
    send_resolved: false  #

route:
  receiver: 'dingtalk'
  routes:
  - match:
      severity: critical
    receiver: 'sms'
    continue: true
  - match:
      severity: warning
    receiver: 'email'
```

8.4 故障排查

8.4.1 常见问题诊断树

```
  "   "


          "RateLimitExceeded" →
          "Timeout" →
          "InternalError" →


              →
             →
          QPS    →
```

→
→

8.4.2 排查命令速查

```
#   Pod
kubectl get pods -n llm-prod
kubectl describe pod <pod-name> -n llm-prod

#
kubectl logs -f deployment/llm-app-deployment -n llm-prod
kubectl logs -f <pod-name> -c llm-app -n llm-prod --tail=1000

#
kubectl exec -it <pod-name> -n llm-prod -- /bin/bash

#
kubectl top pods -n llm-prod
kubectl top nodes

#
kubectl get events -n llm-prod --sort-by='.lastTimestamp'

#
kubectl scale deployment llm-app-deployment --replicas=10 -n llm-prod

#
kubectl rollout undo deployment/llm-app-deployment -n llm-prod
```

8.4.3 应急预案模板

```
#

##
P0 -

##

### 1.    5
- [ ]
- [ ]
- [ ]
```

```
### 2.    15
- [ ]
- [ ]
- [ ]

### 3.    30
- [ ]
- [ ]
- [ ]

### 4.
- [ ]
- [ ]
- [ ]

### 5.
- [ ]
- [ ]
- [ ]
```

本章小结

本章系统讲解了大模型应用的部署与运维：

1. **容器化部署**：Docker 打包、Compose 编排、Kubernetes 集群管理
2. **CI/CD 流程**：GitHub Actions 自动化构建、测试、部署
3. **发布策略**：蓝绿部署、金丝雀发布、自动回滚
4. **监控告警**：日志收集、指标监控、多级告警通知
5. **故障排查**：诊断树、命令速查、应急预案

稳定的运维体系是大模型应用持续服务用户的保障。技术决策者应重视运维能力建设，将 SRE 理念融入日常工作中。

延伸阅读

1. Kubernetes 官方文档
2. 《Site Reliability Engineering》- Google
3. Prometheus 监控最佳实践
4. 阿里云 ACK 服务文档：https://help.aliyun.com/product/44857.html

下一章预告：第 9 章将介绍行业解决方案与实践案例，包括智能客服、企业知识库、代码助手、营销文案

第 9 章 行业解决方案与实践案例

本章导读

理论最终要服务于实践。本章将介绍大模型在各行业的典型应用场景和落地案例，包括智能客服、

核心议题： - 智能客服系统实战 - 企业知识库问答 - AI 代码助手开发 - 营销内容生成 - 教育培训应用 - 医疗健康场景探索

9.1 智能客服系统

9.1.1 场景概述

业务痛点： - 人工客服成本高，培训周期长 - 夜间和节假日服务覆盖不足 - 常见问题重复回答，效率低 - 客户等待时间长，体验差

大模型价值： - 7×24 小时不间断服务 - 同时处理海量并发咨询 - 准确理解用户意图 - 无缝转接人工客服

9.1.2 系统架构

```
(App/ /          (   )          (RAG)
 Web/ )

                    (   )          (Qwen)
```

9.1.3 核心实现

```python
class IntelligentCustomerService:
    def __init__(self):
        self.intent_classifier = self.load_intent_model()
        self.knowledge_base = KnowledgeBase()
        self.llm = QwenClient()
```

```python
def handle_request(self, user_message, user_info):
    # 1.
    intent = self.intent_classifier.predict(user_message)

    # 2.
    if intent == 'greeting':
        return self.handle_greeting(user_message)

    elif intent == 'order_query':
        return self.handle_order_query(user_message, user_info)

    elif intent == 'refund_request':
        return self.handle_refund(user_message, user_info)

    elif intent == 'product_consult':
        # RAG   +
        contexts = self.knowledge_base.search(user_message, top_k=3)
        answer = self.llm.generate_with_context(user_message, contexts)
        return answer

    elif intent == 'complaint':
        #
        return self.transfer_to_human(user_message, user_info)

    else:
        #
        answer = self.llm.generate(user_message)
        if self.is_confident(answer):
            return answer
        else:
            return self.transfer_to_human(user_message, user_info)

def handle_order_query(self, message, user_info):
    """  """
    #
    order_id = self.extract_order_id(message)

    if not order_id:
        #
        orders = self.get_user_recent_orders(user_info['user_id'])
        return f"    {orders}"

    #
    order = self.query_order(order_id)
```

```
        if order:
            return f"""
  {order_id}
  {order['status']}
  {order['logistics']}
   {order['estimated_delivery']}
"""
        else:
            return "          "

    def transfer_to_human(self, message, user_info):
        """   """
        ticket = self.create_support_ticket(message, user_info)

        #
        self.notify_customer_service(ticket)

        return f"""     {ticket['id']}     30
         400-xxx-xxxx """

#
service = IntelligentCustomerService()
response = service.handle_request(
    user_message="      ",
    user_info={'user_id': 'U12345', 'vip_level': 3}
)
print(response)
```

9.1.4 效果评估

某电商客户上线 6 个月数据：

指标	上线前	上线后	提升
自动解决率	35%	78%	+43%
平均响应时间	45 秒	2 秒	-96%
人工客服成本	100 万/月	40 万/月	-60%
客户满意度	4.2	4.6	+9.5%

9.2 企业知识库问答

9.2.1 场景概述

业务痛点： - 企业文档分散，查找困难 - 新员工培训成本高 - 专家经验难以传承 - 重复问题占用大量时间

大模型价值： - 统一知识入口，快速问答 - 降低培训成本 - 沉淀组织智慧 - 释放专家生产力

9.2.2 实施步骤

步骤一：知识梳理

```
#
knowledge_taxonomy = {
    "  ": {
        "  ": ["  A", "  B", "  C"],
        "  ": ["  ", "  ", "  "],
        "  ": ["  ", "  ", "  "]
    },
    "  ": {
        "  ": [" ", " ", " "],
        "  ": [" ", " ", " "],
        "IT  ": ["  ", "  ", "  "]
    },
    "  ": {
        "  ": ["  ", "  ", "  "],
        "  ": [" ", "  ", "  "],
        "  ": ["  ", "  ", "  "]
    }
}
```

步骤二：文档处理流水线

```
class DocumentPipeline:
    def __init__(self):
        self.parsers = {
            '.pdf': PDFParser(),
            '.docx': DocxParser(),
            '.xlsx': ExcelParser(),
            '.pptx': PPTParser(),
            '.md': MarkdownParser()
        }

    def process(self, file_path, metadata):
        # 1.
        ext = os.path.splitext(file_path)[1]
```

```
        parser = self.parsers.get(ext)

        if not parser:
            logging.warning(f"     {ext}")
            return None

        content = parser.parse(file_path)

        # 2.
        if len(content) < 100:
            logging.info(f"     {file_path}")
            return None

        # 3.
        content = self.sanitize(content)

        # 4.
        chunks = self.chunk_document(content, chunk_size=300, overlap=50)

        # 5.
        for chunk in chunks:
            chunk['vector'] = get_embedding(chunk['text'])
            chunk['metadata'] = metadata

        return chunks

    def sanitize(self, content):
        """    """
        #
        content = re.sub(r'\d{11}', '***', content)  #
        content = re.sub(r'\d{18}', '***', content)  #
        return content

#
pipeline = DocumentPipeline()
all_chunks = []

for doc in document_list:
    chunks = pipeline.process(doc['path'], doc['metadata'])
    if chunks:
        all_chunks.extend(chunks)

#
vector_db.upsert(all_chunks)
```

步骤三：问答接口

```
@app.route('/api/knowledge/qa', methods=['POST'])
def knowledge_qa():
    data = request.json
    question = data['question']
    department = data.get('department', 'all')  #

    # 1.
    filters = f"department='{department}'" if department != 'all' else None

    results = vector_db.search(
        query=question,
        top_k=5,
        filter=filters
    )

    # 2.   Prompt
    context = "\n\n".join([
        f"[  {r['metadata']['source']}]\n{r['text']}"
        for r in results
    ])

    prompt = f"""


{context}

  {question}

       """

    # 3.
    response = Generation.call(
        model='qwen3.6-plus',
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0.5
    )

    answer = response.output.choices[0].message.content

    # 4.
    return jsonify({
        'answer': answer,
        'references': [
```

```
            {
                'source': r['metadata']['source'],
                'url': r['metadata'].get('url'),
                'relevance_score': r['score']
            }
            for r in results[:3]
        ]
    })
```

9.2.3 运营优化

持续改进机制：

```
class KnowledgeBaseOptimizer:
    def __init__(self):
        self.feedback_db = FeedbackDatabase()

    def analyze_unanswered_questions(self):
        """        """
        unanswered = self.feedback_db.get_unanswered(limit=100)

        #
        clusters = self.cluster_questions(unanswered)

        #
        suggestions = []
        for cluster in clusters:
            suggestions.append({
                'topic': cluster['topic'],
                'question_count': len(cluster['questions']),
                'sample_questions': cluster['questions'][:5],
                'priority': 'high' if len(cluster['questions']) > 10 else 'medium'
            })

        return suggestions

    def collect_feedback(self, question, answer, rating, comment=None):
        """    """
        self.feedback_db.insert({
            'question': question,
            'answer': answer,
            'rating': rating,  # 1-5
            'comment': comment,
            'timestamp': time.time()
        })
```

```
        #
        if rating <= 2:
            self.notifyKnowledgeOwner(question, answer, comment)

    def generate_improvement_report(self):
        """     """
        feedback_stats = self.feedback_db.get_monthly_stats()

        report = f"""
#

##
-    {feedback_stats['total_questions']}
-    {feedback_stats['avg_rating']:.2f}/5.0
-    {feedback_stats['unanswered_count']}

##    TOP10
{self.format_top_questions(feedback_stats['top_questions'])}

##
{self.format_knowledge_gaps(feedback_stats['gaps'])}

##
1.  {feedback_stats['top_gap_topic']}
2.  {feedback_stats['low_satisfaction_topic']}
3.     {feedback_stats['outdated_docs']}
"""
        return report
```

9.3 AI 代码助手

9.3.1 场景概述

业务痛点： - 重复性编码工作占用大量时间 - 代码审查效率低 - 新人上手慢 - 技术债务积累

大模型价值： - 自动生成样板代码 - 智能补全和建议 - 代码审查和优化建议 - 技术文档自动生成

9.3.2 核心功能

```
class AICodingAssistant:
    def __init__(self):
        self.model = 'qwen3.6-plus'
```

```
    def code_completion(self, prefix_code, language='python'):
        """   """
        prompt = f"""   {language}

{prefix_code}

          """

        response = Generation.call(
            model=self.model,
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.2,  #
            max_tokens=500
        )

        return response.output.choices[0].message.content

    def code_generation(self, requirement, language='python'):
        """     """
        prompt = f"""  {language}

{requirement}


1.
2.
3.
4.

       """

        response = Generation.call(
            model=self.model,
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.3
        )

        return response.output.choices[0].message.content

    def code_review(self, code):
        """   """
        prompt = f"""
1.   bug
2.
```

```
3.
4.
5.


{code}

   5     """

        response = Generation.call(
            model=self.model,
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.3
        )

        return response.output.choices[0].message.content

    def explain_code(self, code):
        """   """
        prompt = f"""

{code}


1.
2.
3.     """

        response = Generation.call(
            model=self.model,
            messages=[{'role': 'user', 'content': prompt}]
        )

        return response.output.choices[0].message.content

    def generate_tests(self, code, framework='pytest'):
        """    """
        prompt = f"""     {framework}

{code}


1.
2.
3.
```

```
4.      80%+"""

        response = Generation.call(
            model=self.model,
            messages=[{'role': 'user', 'content': prompt}]
        )

        return response.output.choices[0].message.content

    def refactor_code(self, code, goal='improve readability'):
        """   """
        prompt = f"""      {goal}


{code}

           """

        response = Generation.call(
            model=self.model,
            messages=[{'role': 'user', 'content': prompt}]
        )

        return response.output.choices[0].message.content

# VS Code
def activate(context):
    assistant = AICodingAssistant()

    #
    context.subscriptions.append(
        commands.register_command('ai.complete', lambda: assistant.code_completion(get_p
    )
    context.subscriptions.append(
        commands.register_command('ai.review', lambda: assistant.code_review(get_selecte
    )
    context.subscriptions.append(
        commands.register_command('ai.explain', lambda: assistant.explain_code(get_selec
    )
```

9.3.3 效果评估

某互联网公司上线数据：

指标	基线	使用后	提升
编码效率	100 LOC/天	180 LOC/天	+80%
Code Review 时间	2 小时/PR	0.5 小时/PR	-75%
Bug 率	5%	3%	-40%
新人上手时间	3 个月	1.5 个月	-50%

9.4 营销文案生成

9.4.1 场景概述

业务痛点： - 营销内容需求量大，人力有限 - 多渠道内容适配工作繁琐 - A/B 测试素材准备周期长 - 品牌调性难以保持一致

大模型价值： - 批量生成高质量文案 - 一键适配多平台风格 - 快速生成 A/B 测试素材 - 保持品牌一致性

9.4.2 实现方案

```
class MarketingCopyGenerator:
    def __init__(self):
        self.brand_voice = self.load_brand_guidelines()

    def load_brand_guidelines(self):
        """    """
        return {
            'tone': '   ',
            'style': '   ',
            'keywords': [' ', ' ', ' '],
            'avoid_words': [' ', ' ', ' '],
            'examples': [...]
        }

    def generate_product_description(self, product_info, platform='taobao'):
        """    """
        platform_styles = {
            'taobao': '     emoji   ',
            'jd': '       ',
            'xiaohongshu': '       ',
            'douyin': '       ',
            'wechat': '       '
        }
```

```
        prompt = f"""     {platform}


-   {product_info['name']}
-   {', '.join(product_info['features'])}
-   {product_info['price']}
-    {product_info['target_audience']}

   {self.brand_voice['tone']}
   {platform_styles.get(platform, '')}


1.     20
2.      100-200
3.
4.     emoji   """

        response = Generation.call(
            model='qwen3.6-plus',
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.7
        )

        return response.output.choices[0].message.content

    def generate_ad_variants(self, base_copy, num_variants=5):
        """       A/B   """
        prompt = f"""      {num_variants}


{base_copy}


1.
2.
3.      / /
4.


  1 ...
  2 ...
..."""

        response = Generation.call(
```

```
            model='qwen3.6-plus',
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.8  #
        )

        variants = self.parse_variants(response.output.choices[0].message.content)
        return variants

    def generate_social_media_posts(self, topic, platforms=['wechat', 'weibo', 'xiaohong
        """       """
        results = {}

        for platform in platforms:
            prompt = f"""  "{topic}"  {platform}


- wechat:
- weibo:
- xiaohongshu:


1.
2.
3.
4.   """

            response = Generation.call(
                model='qwen3.6-plus',
                messages=[{'role': 'user', 'content': prompt}]
            )

            results[platform] = response.output.choices[0].message.content

        return results

#
generator = MarketingCopyGenerator()

#
product = {
    'name': '   X1',
    'features': ['  ', 'GPS  ', 'IP68  ', '7   '],
    'price': '¥999',
    'target_audience': '   '
}
```

```
taobao_copy = generator.generate_product_description(product, platform='taobao')
print(taobao_copy)

#   A/B
variants = generator.generate_ad_variants(base_copy, num_variants=5)

#
social_posts = generator.generate_social_media_posts(
    topic='  11    ',
    platforms=['wechat', 'weibo', 'xiaohongshu']
)
```

9.4.3 效果评估

某品牌电商双 11 实战数据：

指标	传统方式	AI 辅助	提升
文案产出量	50 篇/周	300 篇/周	+500%
单篇成本	200 元	30 元	-85%
CTR 提升	基准	+15%	-
转化率提升	基准	+8%	-

9.5 教育培训应用

9.5.1 场景概述

业务痛点： - 优质教育资源分布不均 - 个性化教学难以实现 - 教师批改作业负担重 - 学习反馈不及时

大模型价值： - 7×24 小时在线辅导 - 因材施教的个性化学习 - 自动批改和反馈 - 降低教育成本

9.5.2 典型应用

1. 智能家教

```
class AITutor:
    def __init__(self, subject='math', grade_level=9):
        self.subject = subject
        self.grade_level = grade_level
        self.student_profile = {}
```

```
    def explain_concept(self, concept):
        """   """
        prompt = f"""     {self.subject}     {self.grade_level}

      "{concept}"
1.
2.
3.
4.


   {self.grade_level}     """

        response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'con
        return response.output.choices[0].message.content

    def generate_exercises(self, topic, difficulty='medium', num_questions=5):
        """   """
        prompt = f"""  {num_questions}  "{topic}"
-   {difficulty}
-     +
-

 {self.grade_level}   """

        response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'con
        return response.output.choices[0].message.content

    def grade_homework(self, student_answer, correct_answer):
        """   """
        prompt = f"""

  {correct_answer['question']}
   {student_answer}
   {correct_answer['answer']}


1.
2.
3.
4.   """

        response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'con
        return response.output.choices[0].message.content
```

```
    def adaptive_learning_path(self, student_performance):
        """     """
        #
        weak_topics = self.analyze_weakness(student_performance)

        #
        prompt = f"""

   {', '.join(weak_topics)}
  2
    30



1.
2.
3.    """

        response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'con
        return response.output.choices[0].message.content
```

2. 作文批改

```
def grade_essay(essay, grade_level, rubric):
    """    """
    prompt = f"""   {grade_level}

   {rubric['title']}

-    30
-    30
-    20
-   20


{essay}


1.
2.
3.
4.
5.     """

    response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'content
    return response.output.choices[0].message.content
```

9.6 医疗健康场景

9.6.1 场景概述

注意：医疗场景需严格遵守相关法规，AI 仅作为辅助工具，不能替代专业医疗诊断。

适用场景： - 健康咨询和科普 - 就诊前分诊引导 - 病历文书辅助撰写 - 医学文献检索

9.6.2 实现要点

```
class HealthAssistant:
    def __init__(self):
        self.disclaimer = "       AI                    "

    def health_consultation(self, symptoms):
        """   """
        prompt = f"""    {symptoms}

  AI
1.       3-5
2.
3.
4.

                """

        response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'con
        return self.disclaimer + "\n\n" + response.output.choices[0].message.content + '

    def triage_guide(self, symptoms, patient_info):
        """   """
        prompt = f"""   {patient_info}
   {symptoms}


1.      /  /   /
2.
3.
4.     """

        response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'con
        return response.output.choices[0].message.content
```

```
    def medical_record_draft(self, patient_history, examination, diagnosis):
        """    """
        prompt = f"""

 {patient_history['chief_complaint']}
 {patient_history['history_of_present_illness']}
 {patient_history['past_medical_history']}
  {examination}
  {diagnosis}

      """

        response = Generation.call(model='qwen3.6-plus', messages=[{'role': 'user', 'con
        return response.output.choices[0].message.content
```

本章小结

本章介绍了大模型在各行业的典型应用场景：

1. **智能客服**：7×24 小时服务，自动解决率提升至 78%，人工成本降低 60%
2. **企业知识库**：统一知识入口，降低培训成本，沉淀组织智慧
3. **AI 代码助手**：编码效率提升 80%，Code Review 时间减少 75%
4. **营销文案**：产出量提升 500%，单篇成本降低 85%
5. **教育培训**：个性化学习，智能批改，促进教育公平
6. **医疗健康**：健康咨询、分诊引导、病历辅助（需严格合规）

技术决策者应结合自身业务特点，选择合适的应用场景，从小规模试点开始，逐步扩大应用范围。

延伸阅读

1. 阿里云百炼行业解决方案：https://www.aliyun.com/solution/bailian
2. 《AI Superpowers》- Kai-Fu Lee
3. Hugging Face 行业案例：https://huggingface.co/case-studies
4. 各垂直领域 AI 应用白皮书

下一章预告：第 10 章将展望大模型应用开发的未来趋势，包括多模态融合、Agent 自主化、边缘部署、隐私计算等前沿方向，帮助技术决策者把握技术演进脉络。

第 10 章 大模型应用开发未来趋势

本章导读

大模型技术正在快速发展。本章将展望未来的技术演进方向，包括多模态融合、Agent 自主化、边缘部署、隐私计算、具身智能等前沿趋势，帮助技术决策者把握技术发展脉络，提前布

核心议题： - 多模态大模型演进 - Agent 自主化与协作 - 边缘计算与端侧部署 - 隐私计算与联邦学习 - 具身智能与机器人 - 可持续发展与绿色 AI

10.1 多模态大模型演进

10.1.1 从单模态到全模态

大模型的发展经历了三个阶段：

阶段一：文本单模态（2018-2022） - GPT-3、BERT 等纯文本模型 - 应用场景：对话、写作、翻译 - 局限性：无法理解图像、音频等其他模态

阶段二：多模态融合（2022-2025） - GPT-4V、Qwen-VL 等视觉 - 语言模型 - 支持图文问答、图像描述 - 特点：多个单模态编码器 + 融合层

阶段三：全模态原生（2025-） - Qwen3.5-Omni、GPT-4o 等原生多模态模型 - 文本、图像、音频、视频统一处理 - 特点：单一架构、端到端、实时交互

10.1.2 技术突破方向

1. 统一表示学习

传统多模态模型使用独立的编码器处理不同模态，然后在高层融合。未来的方向是学习统一的底层表示

```
  Tokenizer →   Transformer →
     ↓
[ ] [ ] [ ] [ ]       token
```

2. 跨模态推理

从简单的”看图说话”升级到复杂的跨模态推理：

```
        +



-
-
-
-
```

3. 实时交互

从”请求 - 响应”模式升级为实时流式交互：

特性	传统模式	实时交互
延迟	秒级	亚秒级
打断	不支持	支持自然打断
多轮	独立请求	上下文连续
情感	无	语气、情绪识别

10.1.3 应用场景展望

1. **智能会议助手** - 实时转录多人对话 - 自动区分发言人 - 理解讨论内容并生成纪要 - 识别行动项并创建待办

2. **沉浸式教育** - 虚拟教师同时讲解和演示 - 实时解答学生问题 - 根据学生表情调整教学节奏 - 生成个性化练习题

3. **全媒体内容创作** - 输入一个创意，自动生成： - 文章 - 配图 - 短视频 - 播客音频 - 保持风格一致性

10.2 Agent 自主化与协作

10.2.1 从工具调用到自主执行

当前 Agent 主要是”人在回路”（Human-in-the-loop）模式：

```
→ Agent    →     → Agent    →
```

未来的发展方向是高度自主：

```
→ Agent     →     →     →
```

自主化程度分级：

等级	名称	特征	示例
L1	辅助执行	人类逐步指令	“调用这个 API”
L2	任务执行	人类给任务，Agent 执行	“帮我订机票”
L3	条件自主	特定场景下自主决策	“出差时自动订票”
L4	高度自主	大部分情况自主，异常请示	“管理我的日程”
L5	完全自主	完全自主，人类只设目标	“提升团队效率”

10.2.2 多 Agent 协作

复杂任务需要多个专业 Agent 协作完成：

组织模式：

```
                Agent (Manager)
    -
    -
    -
```

```
Agent    Agent    Agent
```

协作协议：

```python
class AgentCollaboration:
    def __init__(self):
        self.agents = {
            'manager': Agent(role='  '),
            'researcher': Agent(role='  '),
            'writer': Agent(role=' '),
            'reviewer': Agent(role='  ')
        }

    async def complete_task(self, task_description):
        # 1. Manager
        plan = await self.agents['manager'].plan(task_description)

        # 2.
        research_result = await self.agents['researcher'].research(plan['research_topics

        # 3. Writer
        draft = await self.agents['writer'].write(research_result, plan['outline'])

        # 4. Reviewer
        feedback = await self.agents['reviewer'].review(draft)

        # 5.
        if feedback['needs_revision']:
            revised_draft = await self.agents['writer'].revise(draft, feedback)
```

```
            return revised_draft

        return draft
```

10.2.3 Agent 经济系统

未来可能出现 Agent 之间的价值交换：

- **能力市场**：Agent 可以购买其他 Agent 的服务
- **声誉系统**：基于历史表现建立信用评分
- **激励机制**：完成任务获得奖励，用于购买服务

10.3 边缘计算与端侧部署

10.3.1 云端 vs 边缘

维度	云端推理	边缘推理
延迟	高（网络传输）	低（本地处理）
隐私	数据出域	数据不出设备
成本	按量付费	一次性硬件投入
离线	不可用	可用
模型大小	不限	受限（<1GB）

10.3.2 端侧模型优化

1. 模型压缩

```
(Qwen-7B) → → (Qwen-1.8B)
                ↓
              (INT4)
                ↓
            7GB → 0.5GB
            1x → 5x
```

2. 神经架构搜索（NAS）

自动搜索适合移动设备的模型架构：

```
# NAS
def search_mobile_model(target_latency=50ms, target_accuracy=0.9):
    search_space = {
        'num_layers': [6, 8, 12, 16],
        'hidden_size': [256, 512, 768],
```

```
        'attention_heads': [4, 8, 12],
        'activation': ['ReLU', 'GELU', 'Swish']
    }

    best_model = None
    best_score = 0

    for _ in range(1000):
        #
        config = sample_from_search_space(search_space)

        #
        latency = measure_latency(config, device='mobile')
        accuracy = estimate_accuracy(config)

        #
        score = accuracy if latency <= target_latency else 0

        if score > best_score:
            best_score = score
            best_model = config

    return best_model
```

3. 增量更新

无需重新下载完整模型，仅更新差异部分：

```
  v1.0 (500MB)
 v1.0 → v1.1 (5MB)
```

10.3.3 混合架构

结合云端和边缘的优势：

```
 →
       ↓

       ↓
 →
```

动态路由示例：

```
def hybrid_inference(user_input):
    #
    local_result = local_model.predict(user_input)
```

```
    #
    if local_result.confidence > 0.9:
        return local_result.answer
    elif local_result.confidence > 0.6:
        #
        cloud_result = cloud_model.predict(user_input)
        return cloud_result.answer
    else:
        #
        cloud_result = cloud_model.predict(user_input)
        return cloud_result.answer
```

10.4 隐私计算与联邦学习

10.4.1 数据隐私挑战

大模型训练和使用面临隐私挑战：

- 训练数据可能包含个人隐私
- 用户输入可能泄露敏感信息
- 模型可能记忆并泄露训练数据

10.4.2 联邦学习

核心理念：数据不出本地，模型参数聚合更新。

```
A
        →    →
B                           →    →
        →    →
C
```

应用场景：

- **医疗机构协作**：多家医院联合训练诊断模型，患者数据不出院
- **金融机构反欺诈**：多家银行共享风控模型，客户信息保密
- **输入法优化**：用户打字数据保留在本地，仅上传模型更新

10.4.3 差分隐私

在模型训练或推理中添加噪声，保护个体隐私：

```
def differentially_private_training(data, epsilon=1.0):
    """
```

```
    epsilon
    """
    for batch in data:
        #
        gradients = compute_gradients(batch)

        #
        clipped_gradients = clip(gradients, max_norm=1.0)

        #
        noise = gaussian_noise(scale=1.0 / epsilon)
        noisy_gradients = clipped_gradients + noise

        #
        model.update(noisy_gradients)

    return model
```

10.4.4 可信执行环境（TEE）

使用硬件级别的安全隔离：

```
(      )

                (TEE)



      →    →

(       )
```

阿里云已提供基于 TEE 的机密计算服务，确保数据”可用不可见”。

10.5 具身智能与机器人

10.5.1 从数字世界到物理世界

大模型正在从纯数字领域走向物理世界：

```
2022-2024:
    ↓
2025-2026:
    ↓
2027-:    Embodied AI
```

10.5.2 核心技术

1. 视觉 - 语言 - 动作（VLA）模型

```
  RGB    +
      ↓

      ↓

      ↓
```

2. 世界模型（World Model）

让 AI 理解物理世界的规律：

```
    A    X
      F       Y
```

3. 模仿学习 + 强化学习

```
  1
      ↓

      ↓
  2
      ↓
```

10.5.3 应用场景

1. **家庭服务机器人** - 理解自然语言指令："把客厅桌子上的水杯拿到厨房" - 视觉识别物体和场景 - 路径规划和避障 - 精细操作（抓取易碎物品）

2. **工业制造** - 柔性装配线适应多品种小批量 - 质检：识别微小缺陷 - 预测性维护

3. **医疗护理** - 手术辅助机器人 - 康复训练指导 - 老年照护

10.6 可持续发展与绿色 AI

10.6.1 AI 的碳足迹

大模型的能耗问题日益突出：

模型	训练能耗（MWh）	碳排放（吨 CO_2）
GPT-3	1,287	552
Qwen-Max	~500	~215
一次完整推理	0.001 kWh	0.0004 kg

随着应用规模扩大，推理阶段的能耗远超训练：

```
10    +
 >>
```

10.6.2 绿色 AI 策略

1. 高效模型架构

- MoE（Mixture of Experts）：每次只激活部分参数
- 稀疏注意力：减少计算量
- 早期退出：简单样本提前结束推理

2. 智能调度

```
def green_inference(request):
    #
    if is_peak_hours():
        if request.can_delay():
            schedule_for_off_peak(request)
            return "      "

    #
    region = find_region_with_most_renewable_energy()
    return execute_in_region(request, region)
```

3. 模型共享

- 多个应用共享同一个基础模型
- 减少重复训练和部署
- 云平台提供模型即服务（MaaS）

10.6.3 企业责任

技术决策者应考虑：

1. **碳核算**：追踪 AI 应用的碳排放
2. **绿色采购**：优先选择使用可再生能源的云服务商
3. **效率优化**：持续优化模型效率，减少不必要调用
4. **透明披露**：向利益相关方披露 AI 环境影响

10.7 技术决策者的应对策略

10.7.1 短期（1 年内）

- 掌握现有大模型技术，构建落地应用
- 建立数据基础设施，积累高质量数据
- 培养团队的 AI 工程能力
- 探索 RAG、Agent 等成熟技术

10.7.2 中期（1-3 年）

- 跟踪多模态、边缘部署等技术进展
- 试点端云混合架构
- 建立 Agent 协作框架
- 参与行业标准和生态建设

10.7.3 长期（3-5 年）

- 布局具身智能和机器人应用
- 投资隐私计算和联邦学习
- 构建可持续的 AI 战略
- 培养跨学科人才队伍

本章小结

本章展望了大模型应用开发的未来趋势：

1. **多模态融合**：从文本走向全模态，实现真正的感知 - 认知 - 行动闭环
2. **Agent 自主化**：从辅助执行走向高度自主，多 Agent 协作成为常态
3. **边缘部署**：端侧模型性能提升，混合架构平衡性能与隐私
4. **隐私计算**：联邦学习、差分隐私、TEE 等技术保障数据安全
5. **具身智能**：大模型赋能机器人，从数字世界走向物理世界

6. **绿色 AI**：关注碳排放，推动可持续发展

技术变革的速度超出想象。技术决策者既要拥抱变化，也要保持战略定力，在不确定性中找到确定的方

结语

感谢你阅读完这本书。

大模型技术正在重塑软件开发的范式。作为技术决策者，你站在这一变革的前沿。

本书从基础概念讲起，涵盖了阿里云百炼平台的使用、RAG 和 Agent 开发、安全合规、性能优化、部

但技术只是工具，真正的价值在于解决业务问题。希望你能将书中的知识与自身业务相结合，创造出真
AI 应用。

最后，记住三点：

1. **保持学习**：这个领域变化太快，持续学习是唯一选择
2. **务实落地**：从实际问题出发，避免为 AI 而 AI
3. **承担责任**：技术的力量越大，责任也越大

祝你在 AI 时代的征途中取得成功！

附录 A 术语表

详见 `glossary.md`

附录 B 参考文献

1. 《Attention Is All You Need》- Vaswani et al. (2017)
2. 《Language Models are Few-Shot Learners》- Brown et al. (2020)
3. 《Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks》- Lewis et al. (2020)
4. 《ReAct: Synergizing Reasoning and Acting in Language Models》- Yao et al. (2022)
5. 阿里云百炼官方文档：https://help.aliyun.com/zh/model-studio/
6. 通义千问技术报告：https://qwenlm.github.io/

附录 C 代码示例索引

所有代码示例可在 GitHub 仓库获取： https://github.com/your-username/cloud-llm-book-examples

---

附录 D 学习资源

在线课程： - 吴恩达《AI For Everyone》 - 李宏毅《机器学习》 - Hugging Face NLP Course

技术社区： - 阿里云开发者社区 - Hugging Face 论坛 - LangChain 中文社区

书籍推荐： - 《深度学习》- Goodfellow et al. - 《自然语言处理综论》- Jurafsky & Martin - 《人工智能：一种现代方法》- Russell & Norvig

---

全书完

感谢您完成《云上大模型应用开发实践》的阅读！

术语表

术语表（中英对照）

使用说明: 写作过程中遇到专业术语时，请在此添加并保持全书一致。

A

英文	中文	说明
Agent	智能体	能够感知环境、进行推理并执行动作的自主
API (Application Programming Interface)	应用程序接口	软件系统之间交互的规范
AIGC (AI Generated Content)	人工智能生成内容	使用 AI 技术自动生成文本、图像、音频等内容
AnalyticDB	分析型数据库	阿里云提供的云原生数据仓库服务

B

英文	中文	说明
Batching	批处理	将多个请求合并处理以提高吞吐量
Blue-Green Deployment	蓝绿部署	通过两套环境切换实现零停机发布

C

英文	中文	说明
Chain-of-Thought (CoT)	思维链	引导模型逐步推理的 Prompt 技术
CI/CD (Continuous Integration/Continuous Deployment)	持续集成/持续部署	自动化软件交付流程
CDN (Content Delivery Network)	内容分发网络	分布式网络加速服务
Context Window	上下文窗口	模型一次能处理的 Token 数量上限
Cosine Similarity	余弦相似度	向量空间中衡量两个向量夹角的相似度指标

D

英文	中文	说明
DashScope	灵积模型服务	阿里云提供的大模型 API 服务平台
Docker	容器引擎	应用容器化技术标准

E

英文	中文	说明
ECS (Elastic Compute Service)	弹性计算服务	阿里云虚拟机实例服务
Embedding	嵌入向量	将文本、图像等转换为数值向量的技术

英文	中文	说明
ECI (Elastic Container Instance)	弹性容器实例	阿里云 Serverless 容器服务

F

英文	中文	说明
Few-shot Learning	少样本学习	通过少量示例引导模型理解任务
Function Calling	函数调用	大模型调用外部工具或 API 的能力

G

英文	中文	说明
GPU (Graphics Processing Unit)	图形处理器	并行计算加速芯片，常用于 AI 训练推理
Gray Release	灰度发布	逐步扩大新版本用户比例的发布策略

K

英文	中文	说明
Kubernetes (K8s)	容器编排系统	开源容器集群管理平台

L

英文	中文	说明
LangChain	语言链框架	大模型应用开发框架
LLM (Large Language Model)	大语言模型	基于 Transformer 的大规模语言模型

英文	中文	说明
LlamaIndex	索引框架	用于构建 RAG 应用的数据框架
LoRA (Low-Rank Adaptation)	低秩适配	高效的大模型微调技术

M

英文	中文	说明
MaaS (Model as a Service)	模型即服务	通过 API 提供大模型能力的云服务模式
Multi-modal	多模态	同时处理文本、图像、音频等多种数据类型

N

英文	中文	说明
NPU (Neural Processing Unit)	神经网络处理器	专为 AI 计算设计的芯片

O

英文	中文	说明
OSS (Object Storage Service)	对象存储服务	阿里云提供的海量数据存储服